SKALA

Agile 방법론 및 MSA 개발

2026.08

임성열 Ph.D.

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX



1. Agile 방법론 및 MSA 개발
2. MSA 개발, Sprint Backlog 구현
3. MSA를 이용한 웹 서비스 구축

1. Agile 방법론 및 MSA 개발

- Agile 개요 및 Scrum 핵심 요소
- User Story & 백로그
- 모의 프로젝트 소개

SK 주식회사 AX

Day 1 목차 한눈에 보기

목차	주요 학습내용 (이론 + 실전 보강)	시간	방식
Agile 개요	필요성·비교·적합상황 + 도입 프로세스·실패패턴·전환 체크리스트	1h	이론
Scrum 핵심 요소	구조·산출물 + 이벤트 실전가이드·Sprint 캘린더·Daily 시나리오	1h	이론
User Story & 백로그	정의·원칙 + 그루밍 프로세스·Before/After·Backlog 채우기 가이드	1h	이론
모의 프로젝트 소개	개요 + MSA 아키텍처 상세·컴포넌트 통신흐름·Day1→Day2 연계	1h	이론
모의 프로젝트 가이드	가이드 + 역할배분·샘플 스토리 3종·실행 체크리스트	1h	개념실습
모의 프로젝트 설계 실습	실습 + Planning 진행순서·Task/Point 예시·Board 운영 시나리오	3h	종합실습



백로그 그루밍(Backlog Grooming)은, 스프린트 시작 에 제품 백로그의 항목들을 정리·구체화·우선순위화하는 사전 준비 활동을 의미합니다.

Agile 개요 - 왜 필요한가?

Agile이 필요한 상황

- 시장·고객 요구가 빠르게 변화 → 초기에 요구사항을 100% 확정하기 어려움
- Waterfall은 후반 단계에서 결함/변경 발견 시 수정 비용이 매우 큼
- 짧은 주기로 동작하는 제품을 자주 검증해 리스크를 조기에 낮추는 접근이 필요
- 고객·이해관계자가 개발 중간에도 지속적으로 피드백을 반영할 수 있어야 함

Agile이 잘 맞는 상황

- **요구사항 변동성이 큼**
  시장/고객 니즈가 계속 바뀌는 신규 서비스·스타트업형 프로젝트
- **빠른 피드백이 중요**
  출시 후 사용자 반응을 보며 방향을 조정해야 하는 제품
- **점진적 가치 전달이 가능**
  기능 단위로 쪼개어 순차적으로 릴리즈할 수 있는 구조
- **협업 밀도가 높은 소규모 팀**
  PO·개발팀 간 상시 커뮤니케이션이 가능한 환경

Agile 개요 - 기존 방법론과의 차이

구분	Waterfall	Agile
개발 방식	순차적(설계→개발→테스트) 단계 진행	반복적·점진적(짧은 Sprint 단위 반복)
요구사항	초기에 상세히 확정, 변경 최소화 지향	지속적으로 수집·재우선순위화(Backlog)
고객 피드백	프로젝트 종료 시점에 주로 확인	매 Sprint마다 Review로 상시 확인
산출물 확인 시점	최종 단계에 통합 결과물 확인	매 Sprint마다 동작하는 Increment 확인
변경 대응	변경 비용이 크고 절차가 무거움	변경을 전제로 유연하게 재계획
팀 구조	역할별 분업, 문서 기반 전달	교차기능팀, 상시 협업·짧은 피드백 루프

Agile 도입 프로세스 - 5단계 로드맵

이론을 배우는 것과 실제 팀에 적용하는 것은 다릅니다. 아래 순서로 단계적으로 도입합니다.

1. 현황 진단

기존 프로세스·조직 문화 점검, 병목 구간 파악

2. 파일럿팀 선정

자발적 참여 의사가 있는 1~2개 소규모 팀부터 시작

3. Sprint 0 준비

백로그 초안 작성, Definition of Done 합의, 역할 지정

4. 첫 Sprint 실행

1~2주 짧은 Sprint로 실제 실행, Retro로 즉시 보정

5. 확산 & 정착

성공 사례 공유 후 타 팀으로 점진적 확산

핵심 원칙 : 처음부터 전사 적용을 시도하지 않습니다. 전체 영향이 없는 단위로 시작해 검증하고, 검증 성공 결과를 바탕으로 확산합니다.

실제 적용 시 흔한 문제 상황과 대응

문제 상황	증상	대응 방안
무늬만 Agile	Sprint라는 이름만 쓰고 기존 Waterfall 방식 그대로 진행	이벤트·산출물을 형식이 아닌 목적에 맞게 실제로 운영
역할 겸직 병목	PO가 다른 업무와 겸직하여 의사결정이 지연	PO 전담 배치, 또는 일부 의사결정 권한을 팀에 위임
회고 없는 반복	Sprint만 반복하고 프로세스 개선이 없음	Retrospective 액션 아이템을 다음 Sprint에 반드시 반영
과도한 문서화 병행	Waterfall식 문서 프로세스를 Agile과 병행 유지	꼭 필요한 산출물만 유지, 대화·시각화 중심으로 전환
진행상황 미가시화	Sprint Board 없이 진행 상태를 아무도 모름	Sprint Board·Burndown Chart로 매일 가시화

실행 시나리오 - 우리 팀 Agile 전환 체크리스트

Sprint 0 이전

- PO / SM / Dev 역할을 명확히 확정했는가?
- Definition of Done 초안을 팀과 합의했는가?
- Product Backlog에 최소 10~15개 항목을 확보했는가?

Sprint 0

- Sprint 목표(Goal)를 한 문장으로 정의했는가?
- Sprint Backlog(Task 단위)를 구성했는가?
- Sprint Board(물리 또는 디지털)를 셋업했는가?

첫 Sprint 종료 후

- Retrospective를 진행하고 액션 아이템을 도출했는가?
- Velocity(팀 처리량) 기록을 시작했는가?
- 도출된 개선사항을 다음 Sprint 계획에 반영했는가?

Agile 공정 - Scrum 이란?



https://www.scrum.org/resources/blog/5-scrum-values-take-center-stage

Scrum 구조 및 프로세스

역할 (Role)

Product Owner

제품 백로그 관리, 우선순위 결정, ROI 책임

Scrum Master

프로세스 촉진, 장애물 제거, 팀 보호

Development Team

기능 설계·구현·테스트, Increment 완성

Sprint 프로세스 (반복 주기)

Sprint Planning › **Daily Scrum** › **Sprint Review** › **Retrospective**

다음 Sprint로 반복

Scrum 산출물 (Artifacts)

Product Backlog

제품에 필요한 모든 요구사항의 우선순위화된 목록. PO가 소유·관리하며 지속적 정제(Refinement)됨.

Sprint Backlog

이번 Sprint에서 완료하기로 선택한 백로그 항목 + 이를 구현하기 위한 Task 계획.

Increment

Sprint 종료 시점까지 완료된 모든 백로그 항목의 합.
' 완료 (Done)' 기준 충족 필요.

Definition of Done (완료의 정의) - Increment가 '완료'로 인정받기 위한 팀 공통 기준. 팀별 모의 프로젝트에서도 실습 시작 전 합의 필요.

이벤트별 실전 진행 가이드

이벤트	권장 소요시간	참석자	핵심 아젠다
Sprint Planning	2시간 (2주 Sprint 기준)	전체 팀	목표 합의 → Backlog 선택 → Task 분할
Daily Scrum	15분	Dev팀 + SM	어제 한 일 / 오늘 할 일 / 장애물 공유
Sprint Review	1시간	전체 + 이해관계자	동작하는 결과물 데모, 피드백 수집
Retrospective	45분	전체 팀	Keep / Problem / Try 회고

타임 박스 (Timebox)를 정하고 이것을 지키는 것이 핵심입니다. 논의가 길어지면 별도 회의로 분리하여 해당 이벤트 시간을 지킵니다.

실행 시나리오 - 2주 Sprint 캘린더 예시



하루 일과 예시 (Daily Scrum 기준)

09:30 - 09:45	Daily Scrum - 어제/오늘/장애물 공유
09:45 - 12:00	개발 집중 시간 (Task 작업)
14:00 - 17:00	개발 집중 시간 + 필요 시 Backlog Refinement
17:00 - 17:15	Sprint Board 업데이트 (Task 상태 갱신)

실행 시나리오 - Daily Scrum 대화 예시

팀원 A (백엔드)

어제는 로그인 API 개발을 완료했습니다. 오늘은 토큰 갱신 로직을 진행합니다. 특별한 장애물은 없습니다.

팀원 B (백엔드)

어제는 Gateway 라우팅 설정 작업 중이었고, 오늘 이어서 진행합니다. Kafka 연동 설정 관련해서 SM님과 논의가 필요합니다.

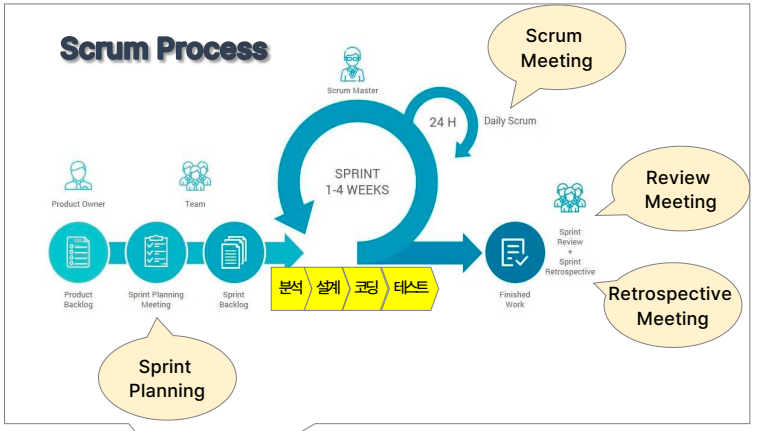
팀원 C (프론트)

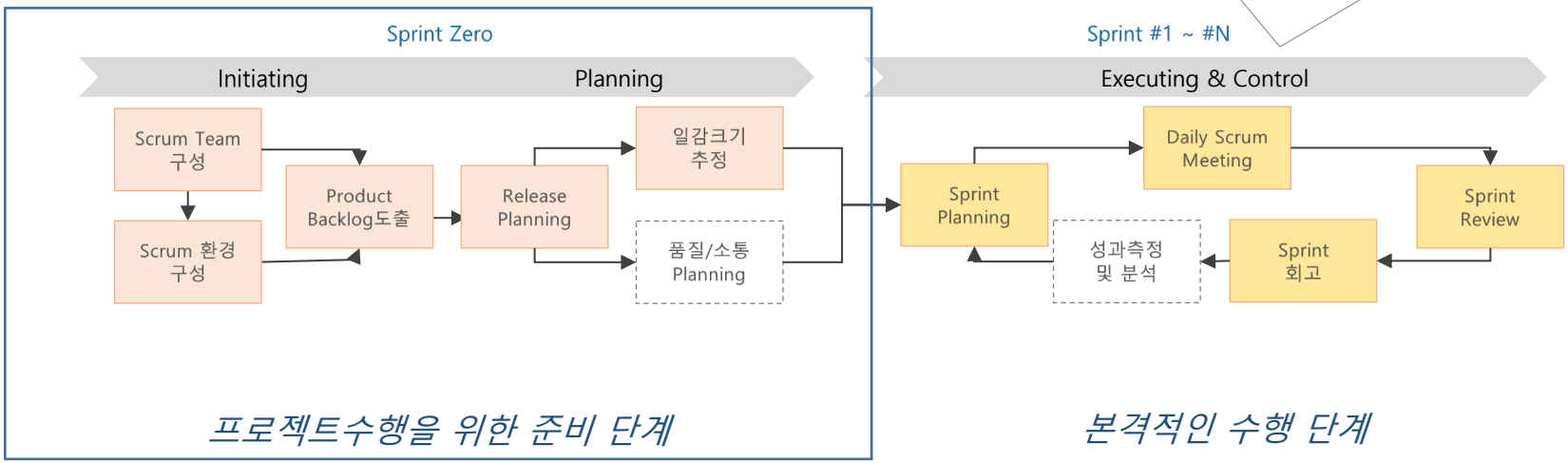
어제는 로그인 화면 UI를 완성했습니다. 오늘은 API 연동을 시작합니다. 팀원 A님의 API 완료 시점 확인이 필요합니다.

⚠ 장애물(예: Kafka 연동, API 완료 시점)은 Daily Scrum에서 발견만 하고, 실제 논의는 종료 후 별도 시간에 진행합니다.

Agile 공정 - Scrum과 Agile Delivery 공정

Agile 공정은 현실적인 SI 프로젝트 수행 방식에 맞게 Scrum Process를 커스터마이징하여, 프로젝트 실행 단계 외에 착수/준비 단계를 지원하기 위한 공정을 추가 (Sprint #0)





User Story - 정의와 방법론상 위치

USER STORY & 백로그 · 1/5

요구사항이 구현 단위로 구체화되는 흐름

비즈니스 요구 > **Epic** > **User Story** > **Task**

Product Backlog는 Epic/User Story 단위로 구성되며, Sprint에 들어온 User Story가 Task로 쪼개져 Sprint Backlog가 됩니다.

작성 템플릿

As a **[사용자 유형]**, I want **[원하는 기능/행동]**, So that **[얻고자 하는 가치/이유]**

예: As a 회원, I want 비밀번호를 재설정하고 싶다, So that 계정 접근 권한을 다시 얻을 수 있다.

User Story 작성 원칙 & 적용 수준

USER STORY & 백로그 · 2/5

INVEST 원칙	의미
Independent	다른 스토리와 독립적으로 개발 가능
Negotiable	세부 구현은 협의 가능(계약이 아닌 대화)
Valuable	사용자 및 고객에게 명확한 가치 제공
Estimable	팀이 규모를 추정할 수 있을 정도로 명확
Small	한 Sprint 내 완료 가능한 크기
Testable	완료 여부를 검증할 수 있는 기준 존재

실전 적용 수준 - 얼마나 상세히 쓰는가?

- **백로그 초기 등록 시**
  제목 + 한 줄 가치 정도로 개략적으로 기록 (Epic 수준)
- **Backlog Refinement 시점**
  Sprint 진입 후보로 좁혀지면 팀과 함께 세부 조건·인수 기준 논의
- **Sprint Planning 직전**
  INVEST 기준 충족하도록 상세화 - Task 분할이 가능한 수준까지
- **Sprint 진행 중**
  필요 이상으로 앞서 상세화하지 않음 (Just-in-time 원칙)

백로그 그루밍(Refinement) 실전 프로세스

USER STORY & 백로그 · 3/5

주기

Sprint 중반 1회, 1시간 이내

참석자

PO 필수 참석, Dev팀 일부 순환 참석

목표

다음 1~2 Sprint 분량의 Story가 'Ready' 상태로 유지

진행 순서

1. **우선순위 재정렬**
   PO가 비즈니스 가치 기준으로 Backlog 순서를 재검토
2. **상위 항목 상세화**
   Sprint 진입 후보 Story의 세부 조건·인수기준을 팀과 함께 논의
3. **INVEST 체크**
   각 Story가 Independent/Negotiable/Valuable/Estimable/Small/Testable을 충족하는지 점검
4. **Story Point 추정**
   Planning Poker 등으로 팀이 함께 규모를 추정

실전 작성 Before / After 비교

USER STORY & 백로그 · 4/5

Before - 모호한 작성

"로그인 기능 개발"

- 누가(사용자 유형) 사용하는지 불명확
- 왜(가치) 필요한지 이유가 없음
- 완료 기준(인수조건)이 없어 검증 불가
- 팀마다 다르게 해석할 위험이 큼

After - INVEST 적용

As a 회원, I want 이메일/비밀번호로 로그인하고 싶다, So that 내 계정에 안전하게 접근할 수 있다

인수 기준 (Acceptance Criteria)

- 올바른 이메일/비밀번호 입력 시 메인 화면으로 이동한다
- 잘못된 정보 입력 시 오류 메시지를 표시한다
- 5회 연속 실패 시 계정이 일시 잠긴다

실행 시나리오-팀 Backlog 채우기 가이드

USER STORY & 백로그 · 5/5

1. **기능 브레인스토밍 (Epic 단위)**
   팀 전체가 모여 제품에 필요한 큰 기능 덩어리를 나열

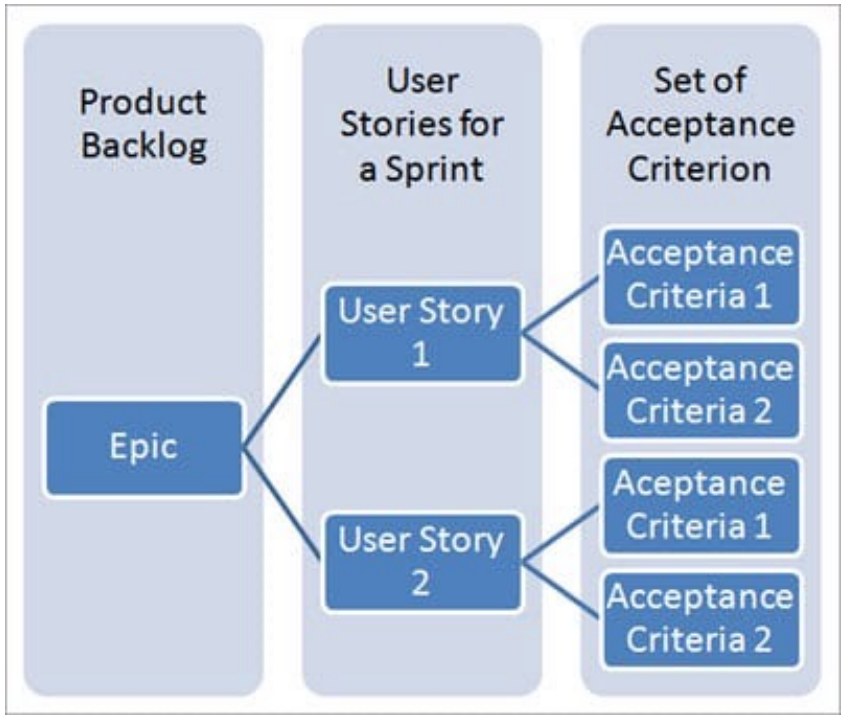
2. **Epic을 User Story로 분해**
   각 Epic을 "As a ~, I want ~, So that ~" 형식의 작은 단위로 쪼갬

3. **MoSCoW로 우선순위 태깅**
   Must / Should / Could / Won't 기준으로 각 Story에 우선순위 부여

4. **상위 10개 INVEST 체크 및 상세화**
   우선순위 상위 항목부터 INVEST 기준으로 다듬고 인수기준 작성

5. **Planning Poker로 추정**
   팀이 함께 Story Point를 추정하여 규모에 대한 공통 이해 형성

Agile 공정 - 제품 백로그와 완료 기준 (1/2)

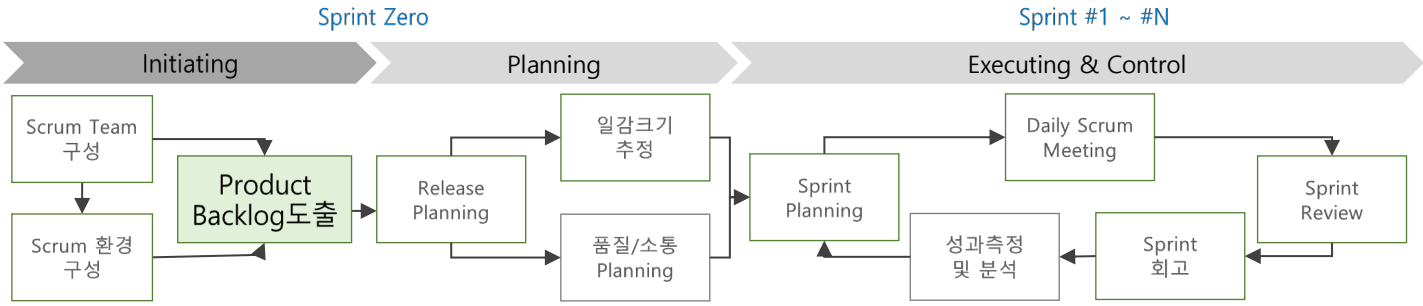


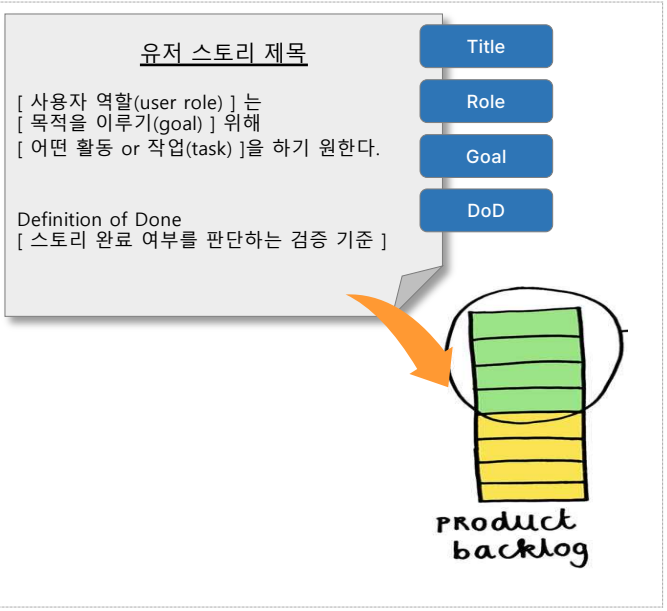
- **제품 백로그(Product Backlog) :** 개발할 제품 요구사항인 사용자 스토리 집합이며, 우선순위로 관리
- **사용자 스토리(User Story)** : 과거 요구사항 명세처럼 업무 범위를 구체화하기 위한 개발자 입장이 아닌, User Story는 사용자가 사용하는 관점에서 어떤 가치를 제공할 것인지를 설명 (해설 : PO는 이 기능이 누구에게 무슨 value를 제공하는지를 설명하고, 향후 개발자는 그 기능의 Value를 제공하기 위한 기술적인 역할과 책임을 가짐)
- **완료 기준(Definition of Done), 인수 기준(Acceptance Criteria)** : 사용자 스토리를 완료시키기 위한 조건 명세(Given, When, Then)

https://www.softwaretestinghelp.com/user-story-acceptance-criteria/

Agile 공정 - 제품 백로그와 완료 기준 (2/2)

요구사항으로부터 사용자 스토리와 제품 백로그를 만드는 과정

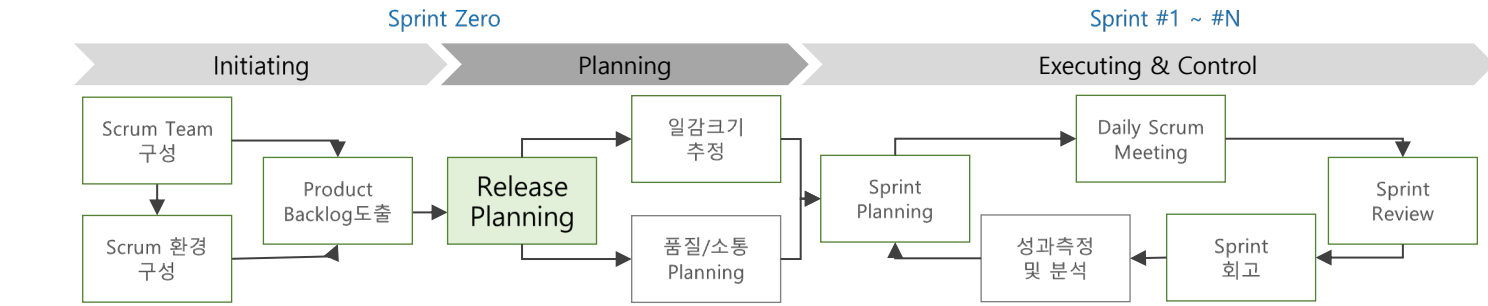


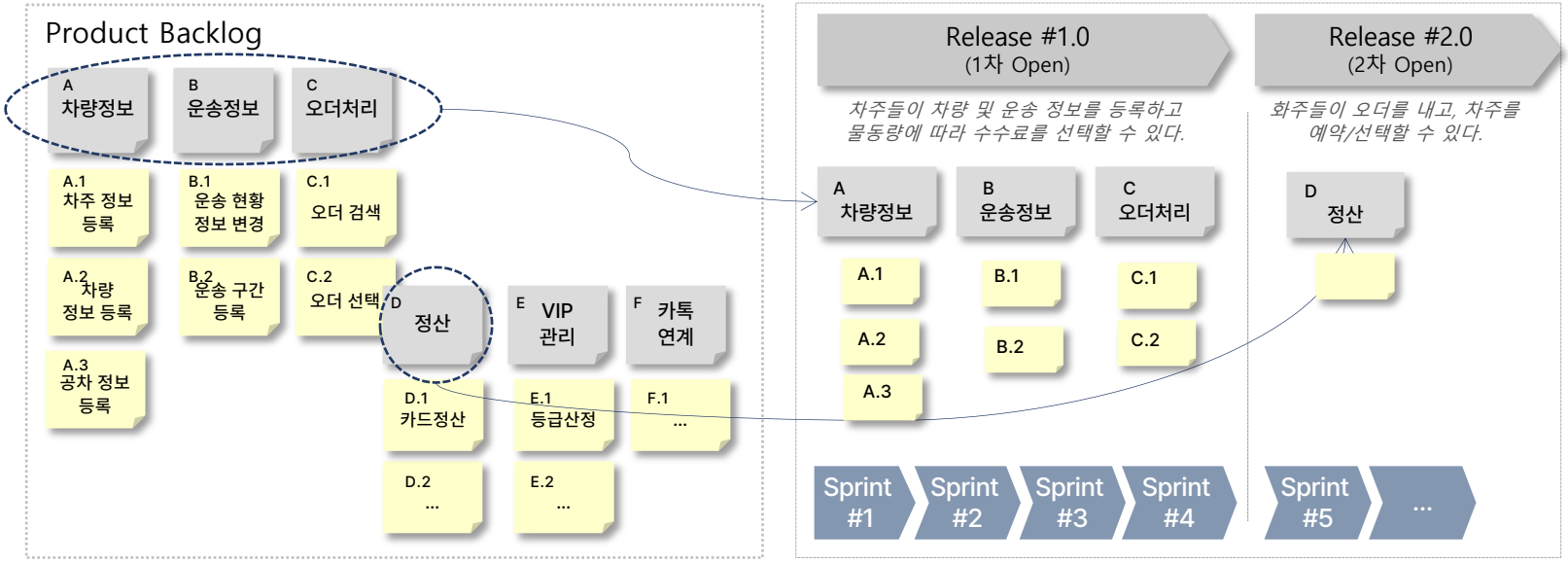


- **사용자 이야기(User Story)**
  - 사용자 관점에서 짧게 기술된 기능에 대한 설명
  - 제품에 담고자 하는 기능 및 요구사항이라 볼 수도 있음
  - 제품과 관련하여 해야 할 일을 의미하는 것은 무엇이나 제품 백로그에 포함됨(제품 특징, 기능, 기술, 개선사항, 오류 등)
- **제품 백로그(Product Backlog)**
  - 우선순위가 부여된 User Story의 집합체 목록
  - **제품 백로그의 Ownership은 제품 책임자가 가짐**
- **요구사항 ≒ 사용자스토리 ≒ 백로그 ≒ 일감**
- **팀이 일하는 근거는 백로그가 되며, 모든 커뮤니케이션 및 성과, 측정은 백로그를 중심으로 이루어짐**

Agile 공정 - Release Planning (1/4)

릴리즈 계획은 백로그의 우선순위에 기반하여 제품의 출시 계획 및 스프린트 일정을 수립





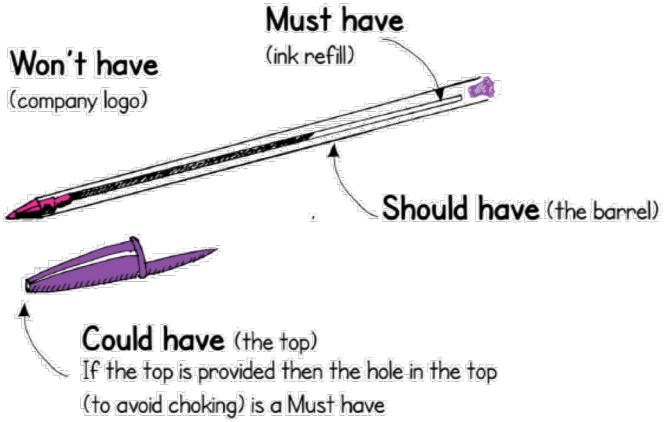
Agile 공정 - Release Planning (2/4)

비즈니스 우선순위는 고객 관점에서 MoSCoW원칙 또는 User 행동기법을 참고하여 결정

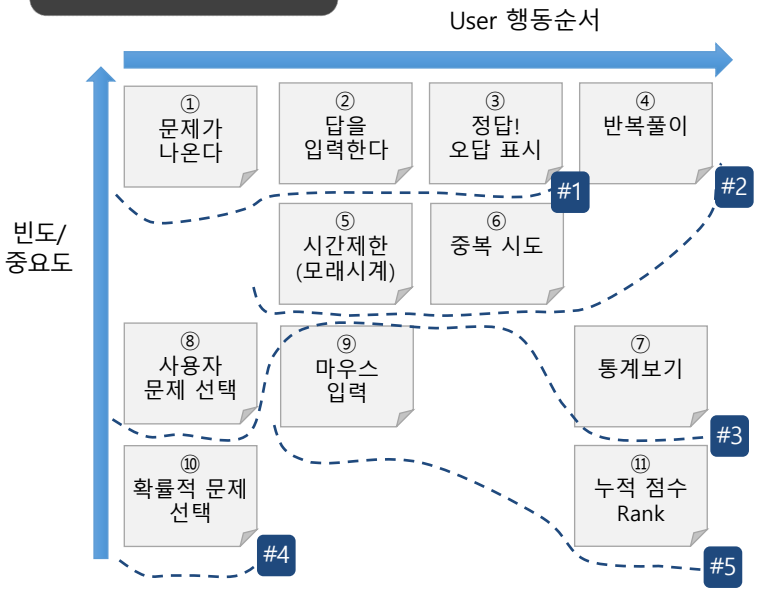
비즈니스 우선순위 정의 → Release Roadmap 수립 → Sprint 일정 수립

- 비즈니스 우선순위 정의
- Release별 Epic 배치
- Sprint 주기 결정

MoSCoW 기법



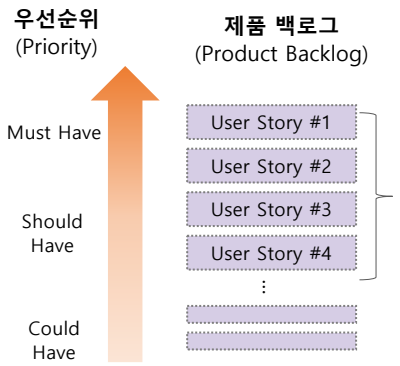
User 행동 기법

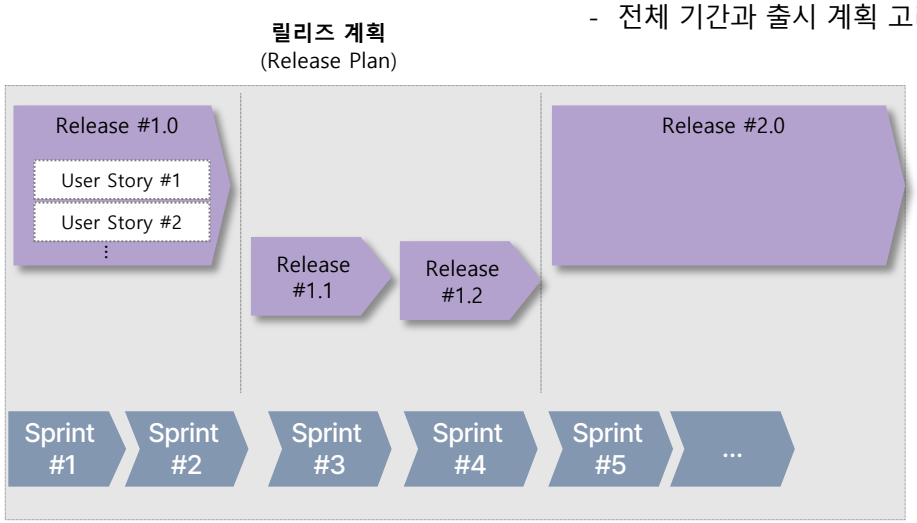


Agile 공정 - Release Planning (3/4)

비즈니스 우선순위 정의 → Release Roadmap 수립 → Sprint 일정 수립

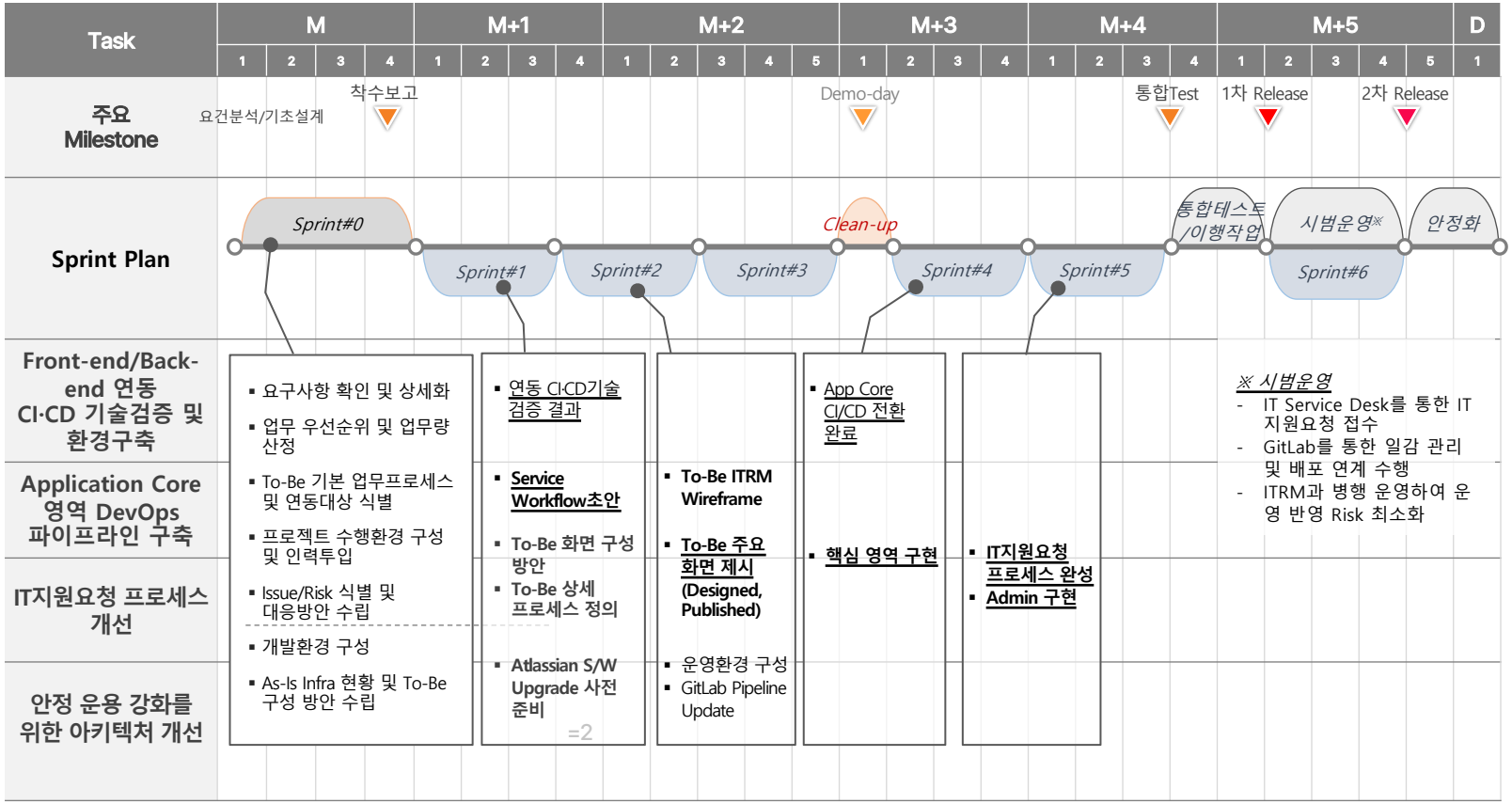
- 비즈니스 우선순위 정의
- Release별 Epic 배치
- Sprint 주기/회차 정의
  - Sprint 주기는 2~4주 사이
  - 전체 기간과 출시 계획 고려





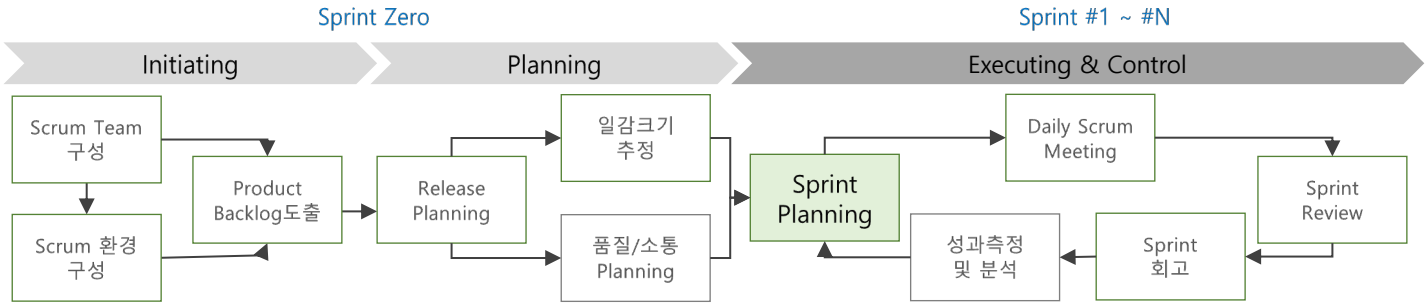
Agile 공정 - Release Planning (4/4)

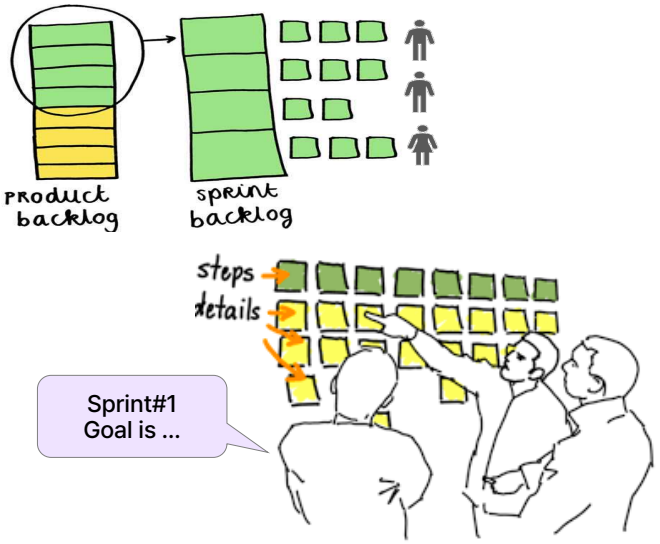
프로젝트 이해 관계자와 공유하기 위한 Release Plan을 다음과 같이 작성



Agile 공정 - Sprint Planning (1/2)

스프린트에서 계획은 스프린트 목표와 스프린트 백로그를 구체적으로 정의하는 과정



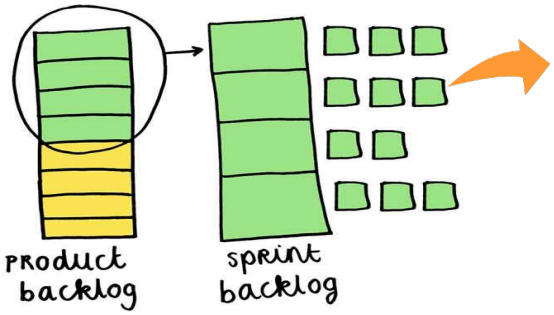


- **Sprint Planning Meeting에서는...**
  - 스프린트의 목표를 수립한다.
  - 스프린트 목표에 필요한 제품 백로그를 선정 후,
  - 목표 달성(백로그 완료)를 위한 작업(Task/Activity)을 상세화한다.
  - 개발팀 구성원은 각자의 역량에 따라 수행할 작업을 할당하고,
  - 스프린트 주기와 일감 크기를 고려하여 작업 완료 일정을 계획한 다음,
  - 스크럼 팀원들과 함께 스프린트 백로그 및 일정을 공유한다.

- **주의사항**
  - 스크럼 팀은 공동으로 작업하며, 주도적, 자율적으로 일감을 선택한다.
  - 작업량이 너무 많거나 너무 적을 경우, 개발팀은 제품 책임자와 제품 백로그 항목들을 재협상할 수 있다.
  - Sprint Planning 시간은 스프린트 주기(2~4주)에 따라 4~8시간을 넘지 않도록 한다.

Agile 공정 - Sprint Planning (2/2)

스프린트 백로그는, 제품 백로그 중 금번 스프린트를 위해 선택된 백로그 및 상세화된 작업 내역 의미



- *백로그 내용이 충분하지 않다면?*
  → 제품 책임자와 함께 User Story를 먼저 구체화해야 함

HTML 편집 기능 20

[user role : 블로그 작성자] 는
[goal : 블로그 포스팅] 을 하기 위해
[task : HTML 편집 기능]를 원한다.

Definition of Done :
- HTML5를 지원해야 한다.
- Tag 자동 작성 기능이 제공되어야 한다.
- 편집기에서 저장된 후, Syntax는 자동 체크된다.
- Preview 가 제공되어야 한다.

HTML 편집 기능 9

[user role : 블로그 작성자] 는
[goal : 블로그 포스팅] 을 하기 위해
[task : HTML 편집 기능]를 원한다.

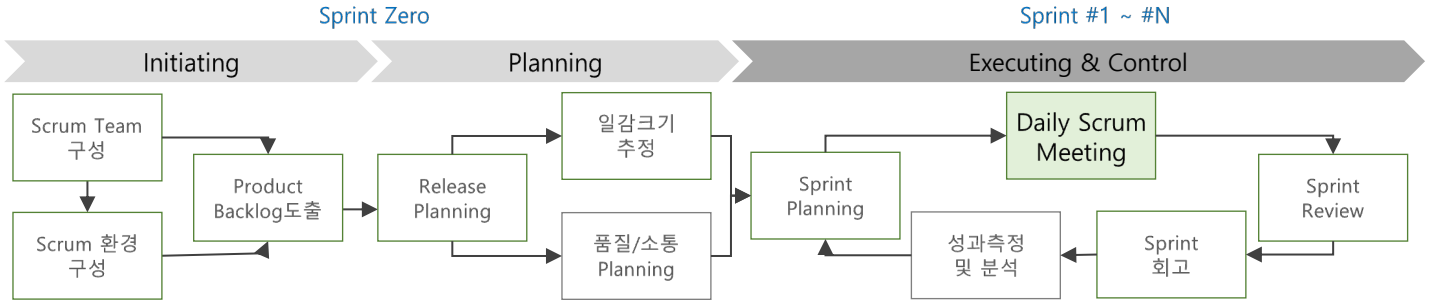
Definition of Done :
- Preview 가 제공되어야 한다.
- Preview시 HTML5를 지원해야 한다.
- Tag 입력시, 기존 Tag가 자동으로 선제공
- 저장 버튼 클릭시 Syntax 체크하여 Warning

Backlog Item (=User Story)

- 웹 에디터 (오픈소스) 후보 조사
- 웹 에디터 검증 및 선정
- 웹 에디터 커스터마이징
- 태그 입력 및 저장 기능 개발
- Warning 창 보여주기 개발

Agile 공정 - Daile Scrum

일일 스크럼은 스크럼 팀의 집중도를 높이고, 팀원간 유대와 신뢰를 형성하는 것을 지원





- ✓ 나는 어제 하루동안 개발팀의 스프린트 목표달성을 위해 무엇을 했는지?
- ✓ 나는 오늘 하루동안 개발팀의 스프린트 목표달성을 위해 무엇을 할 것인지?
- ✓ 나 혹은 개발팀이 스프린트 목표 달성을 하는데 방해요소가 있는지?

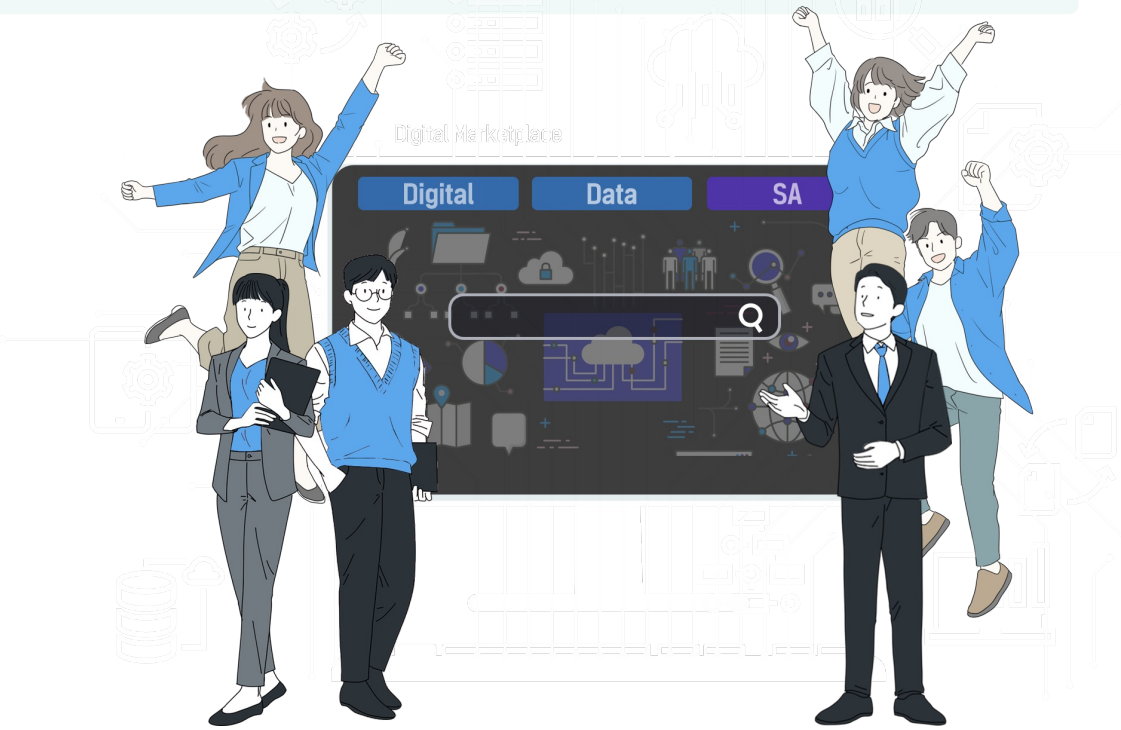
- **일일 스크럼 특징**
  - 매일 같은 시간에 수행하며, 15분을 넘지 않는다.
  - Time Box를 지키기 위해 서서 진행한다. (Stand-Up Meeting)
  - 한 일, 할 일, 장애물 중심으로 돌아가면서 이야기한다.
    (작업 상태 공유를 위해 스프린트 보드 등을 이용하기도 한다)

- **주의사항**
  - 한 사람이 발언권을 독점하지 않는다.
  - 특정 이슈에 대해서 해결책 논의가 필요할 경우, 별도 회의 시간(SoS Meeting) 을 요청한다.
  - 일일 스크럼이 원활히 진행될 수 있도록 스크럼 마스터의 역할이 중요하다.

모의 프로젝트 소개

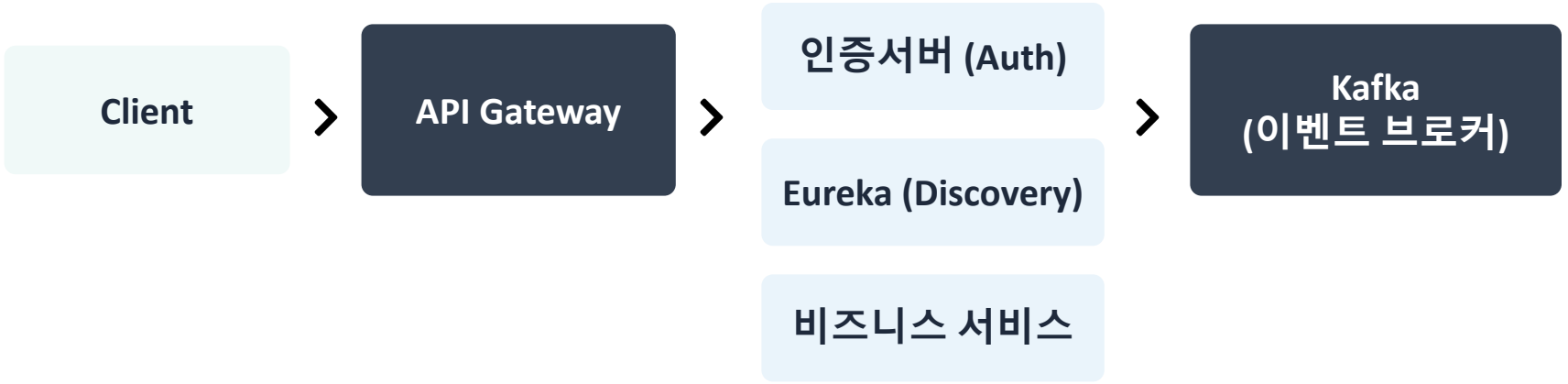
- Day2에 구현할 MSA 시스템 전체 아키텍처 개요
- 핵심 컴포넌트 : 인증서버 / 유레카(Eureka) / Gateway / Kafka 기반 서비스
- Day1 산출물(Sprint Backlog)이 Day2 구현으로 이어지는 흐름 안내
- 다음 슬라이드에서 아키텍처와 컴포넌트별 역할을 상세히 다룹니다.





MSA 아키텍처 상세

모의 프로젝트 소개 · 1/3 · (아키텍처)



요청 흐름 예시

- ① Client가 API Gateway로 요청을 전송 (모든 요청의 단일 진입점)
- ② Gateway가 Eureka에서 대상 서비스 위치를 조회 후 라우팅
- ③ 인증이 필요한 요청은 인증서버에서 JWT 토큰을 검증
- ④ 비즈니스 서비스 처리 후, 상태 변경 이벤트를 Kafka로 발행(다른 서비스가 비동기로 구독)

컴포넌트별 역할과 통신 흐름

모의 프로젝트 소개 · 2/3 · (아키텍처)

컴포넌트	역할	통신 방식
인증서버 (Auth)	로그인 처리, JWT 토큰 발급·검증	REST (동기)
Eureka	서비스 등록 및 위치 탐색(Discovery)	REST 등록, 클라이언트 조회
API Gateway	모든 요청의 진입점, 라우팅·인증 필터	REST (동기)
Kafka	서비스 간 이벤트 기반 비동기 통신	Producer / Consumer, Topic 기반

동기(REST) vs 비동기(Kafka) 선택 기준 : 즉시 응답이 필요하면 REST, 서비스 간 느슨한 결합·이벤트 전파가 목적이면 Kafka를 사용합니다.

실행 시나리오 - Day1 → Day2 연계

모의 프로젝트 소개 · 3/3 · (아키텍처)

Day1에서 작성한 User Story (예시)	Day2 구현 컴포넌트
As a 회원, I want 로그인하고 싶다, So that 내 계정에 접근할 수 있다	인증서버 (Auth) API 구현
As a 서비스, I want 다른 서비스를 찾고 싶다, So that 요청을 라우팅할 수 있다	Eureka 등록/조회 연동
As a 사용자, I want 주문 상태 변경을 실시간으로 알고 싶다, So that 대기 없이 확인할 수 있다	Kafka 이벤트 발행/구독 구현

Day1에 작성한 User Story의 인수기준이 Day2 구현의 테스트 기준이 됩니다. 상세할수록 Day2 실습이 원활합니다.

모의 프로젝트 가이드

모의 프로젝트 가이드 · 1/4

- 팀별로 모의 프로젝트 시스템에 대한 User Story 작성 가이드 제공
- 팀 구성 및 역할 배분 (PO / SM / Dev 지정)
- 실습 전 유의사항 및 산출물 양식 안내
- 다음 슬라이드에서 역할 배분, 샘플 스토리, 실행 체크리스트를 상세히 다룹니다.



팀 구성 및 역할 배분 가이드

모의 프로젝트 가이드 · 2/4 · 상세

역할	배정 기준	책임
Product Owner (1명)	팀 내 우선순위 결정 경험자 또는 자원자	User Story 우선순위 결정, 인수기준 확정
Scrum Master (1명)	프로세스 진행을 도울 수 있는 인원	타임박스 관리, 진행 촉진, 장애물 정리
Development Team (나머지 전원)	구현을 담당할 팀원	Task 수행, 추정 참여, 상호 리뷰

역할은 실습 중 고정하지 않아도 됩니다. 다음 Sprint(Day2)에서는 다른 역할을 경험해 보는 것을 권장합니다.

User Story 작성 실전 가이드 - 샘플 스토리 3종

모의 프로젝트 가이드 · 3/4 · 상세

인증 관련

As a 회원, I want 이메일/비밀번호로 로그인하고 싶다, So that 내 계정에 접근할 수 있다.

Gateway 관련

As a 시스템 운영자, I want 모든 요청이 Gateway를 거치게 하고 싶다, So that 인증을 한 곳에서 일관되게 처리할 수 있다.

Kafka 관련

As a 사용자, I want 주문 상태 변경을 실시간 알림으로 받고 싶다, So that 진행 상황을 바로 알 수 있다.

팀별로 위 예시를 참고해 담당 컴포넌트(인증서버/Eureka/Gateway/Kafka)별로 최소 2개 이상의 User Story를 작성합니다.

실행 시나리오 - 가이드 세션 진행 체크리스트

모의 프로젝트 가이드 · 4/4 · 실행 시나리오

1. **팀 킥오프 (10분)**
   팀원 소개, 역할(PO/SM/Dev) 배분

2. **도메인 이해 (15분)**
   모의 프로젝트 MSA 시스템 개요 재확인, 질의응답

3. **Epic 도출 (15분)**
   인증/Gateway/Kafka 등 컴포넌트 단위로 큰 기능 나열

4. **User Story 작성 (15분)**
   Epic별 최소 2개 이상 User Story를 템플릿에 맞춰 작성

5. **팀 내 리뷰 (5분)**
   작성한 Story가 INVEST 기준을 충족하는지 상호 점검

모의 프로젝트 설계 실습

모의 프로젝트 설계실습 · 1/4

STEP 1

Sprint Planning 실습

- 팀별 작성 User Story에 Story Point 부여
- Sprint Backlog 구성
- Task 단위로 분할 (예: "인증서버 API 구현", "Kafka 메시지 발행 구현" 등)

STEP 2

Daily Scrum 시뮬레이션 + Board 구성

- Task를 칸반보드 (To Do / In Progress / Done)에 배치
- 팀별 Daily Scrum 형식(일일 이슈) 데모 시연
- Day2 작업 분배 논의

Sprint Planning 실전 진행 순서

모의 프로젝트 설계실습 · 2/4 · 상세

단계	소요시간	진행 내용
목표 합의	20분	PO가 이번 Sprint에서 달성할 목표를 한 문장으로 제안, 팀이 합의
Backlog 선택	30분	우선순위 상위 User Story 중 이번 Sprint에 넣을 항목 선택
Task 분할	50분	선택된 Story를 구현 가능한 Task 단위로 쪼갬
추정 & 배분	20분	Task별 담당자·예상 소요시간을 팀이 함께 배분

실습에서는 위 시간 배분을 절반으로 축소해 진행합니다 (총 60분 내외). 핵심은 순서를 지키는 것입니다.

Task 분할 & Story Point 추정 예시표

모의 프로젝트 설계실습 · 3/4 · 상세

User Story	Task	Point
로그인 API 구현	인증서버 엔드포인트 개발	**3**
	JWT 토큰 발급 로직 구현	**2**
	단위 테스트 작성	**1**
Kafka 메시지 발행	주문 상태 변경 이벤트 Producer 구현	**3**
	Topic/Consumer 그룹 설정	**2**

Story Point는 절대적 시간이 아닌 상대적 난이도(피보나치: 1,2,3,5,8 등)로 추정합니다. 팀 내 기준을 먼저 합의하세요.

Sprint Board 운영 & Daily Scrum 시나리오

모의 프로젝트 설계실습 · 4/4 · 실행 시나리오

To Do	In Progress	Done
Kafka Topic 설정	로그인 API 구현	Gateway 라우팅 설정
단위 테스트 작성	JWT 토큰 로직	

Daily Scrum 시나리오

"어제 Gateway 라우팅 설정을 완료해서 Done으로 옮겼습니다.
오늘은 로그인 API 구현을 시작하는데, JWT 관련 라이브러리 버전 확인이 필요해서 In Progress에 유지하겠습니다."

실행 로드맵 - 백로그 도출 → Sprint 분할 → Scrum 진행

DAY 1 → DAY 2 · 실행 로드맵

Day1 에서 만든 산출물이 실제로 어떻게 실행되는지, 전체 흐름을 하나로 연결합니다.

① 백로그 도출

Day 1

Epic → User Story 작성, MoSCoW 우선순위화, Story Point 추정 → Product Backlog 확정

② Sprint 분할

Day 1 → Day 2 경계

확정된 Backlog 중 이번 실습은 Sprint 단위로 나눔 → Sprint Goal/ Sprint Backlog 확정 (Sprint Planning)

③ Scrum 진행

Day 2

Sprint Backlog 기준으로 Daily Scrum → 구현 → Sprint Review(데모) → Retrospective 순으로 실행

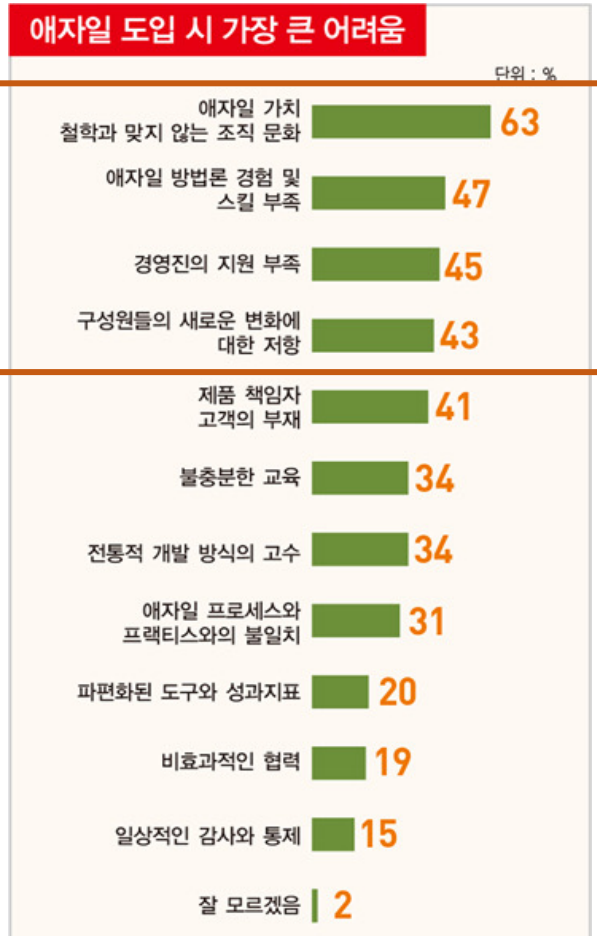
구분	목표	주요 활동
Sprint 0 (Day1)	백로그 정리 + Sprint 계획	User Story·Backlog 확정, Sprint Backlog·Task 분할
Sprint 1 (Day2)	Scrum 사이클 실행	Daily Scrum → 구현 → Sprint Review → Retrospective

Day 1 마무리 · Day 2 예고

🏁 Day1에서 완성한 Sprint Backlog는 Day2 MSA 개발 실습의 기반이 됩니다.



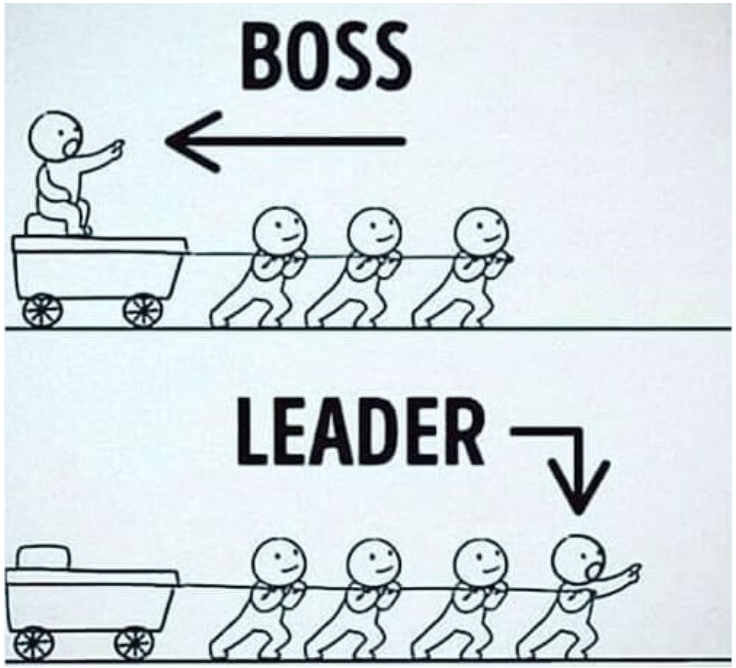
Agile 적용 - 실패 사유



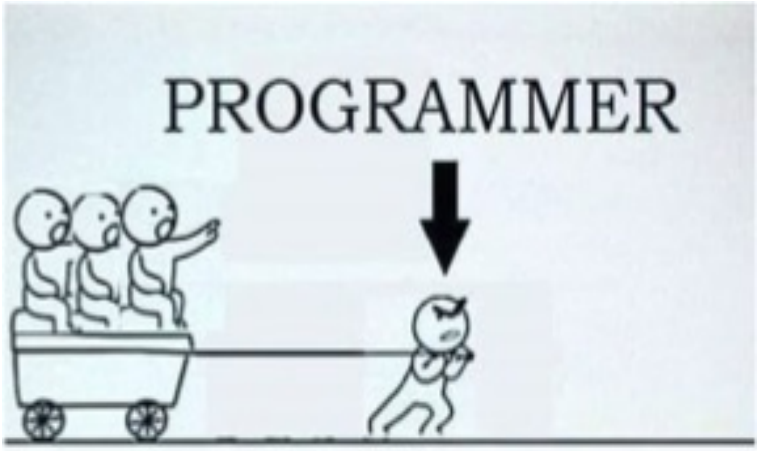
Agile 가치와 철학을 실현하려는
조직, 팀의
의지와 지원이 중요

출처 : State of Agile Report 2016, 2018
https://m.post.naver.com/viewer/postView.nhn?volumeNo=17832865

Agile 적용 - 자율적이고 주도적인 구성 (1/2)



Boss와 Leader의 차이



현실

2. MSA 개발, Sprint Backlog 구현

- MSA (Microservice Architecture) 개요
- 모놀리식 인증/로그인 → SOLID 적용 후 구조
- 인증 서버 & API Gateway
- 서비스 디스커버리 (Eureka) & 서비스간 통신
- Kafka 기본
- Sprint 실행 및 Task 구현
- Sprint Review & Restrospective

SK 주식회사 AX

DAY 2 · 시작하기

MSA 개발

Agile Sprint로 나누어 실행하는 인증/로그인 MSA 전환 + Eureka + Kafka + 수강신청 데모

교육 기간

2일 · 16시간 (Day2)

교육 방식

이론 50% · 실습 50%

Day2 목표

Sprint 실행 · Task 구현

교육 개요

교육 목표

Day 1 : Agile 방법론 & Sprint 운영
이론 4h + 실습 4h - Sprint Backlog 구현

Day 2 : MSA 개발
Day1의 Agile 방법론으로 MSA 개발을 Sprint로 나누고, 각 Sprint 안에서 Scrum을 실행하며 인증/로그인 → Eureka → Kafka → 수강신청 데모까지 구현

Day2 핵심 산출물

Pure Java 인증/로그인을 SOLID 적용 후 MSA로 분할, Spring Boot 기반 OAuth 인증서버 + Eureka + Kafka 연동 완료

Day 2 목차 한눈에 보기

목차	주요 학습내용 (이론 + 실전 보강)	시간	방식
MSA 개요	모놀리식과의 차이 + Agile 기반 Sprint 분할 로드맵 + SOLID 리팩토링	1h	이론
인증서버 & API Gateway	JWT/Gateway 이론 + Sprint1 Planning + OAuth 인증서버 구현 프로세스 + 실행 데모	1h	이론/실습
서비스 디스커버리(Eureka) & 서비스 간 통신	Eureka 이론 + Sprint2 Planning + 연동 프로세스 + 실행 시나리오	1h	이론/실습
Kafka 기본	Kafka 이론 + Sprint3 Planning + Producer/Consumer 구현 + 수강신청 이벤트 데모	1h	이론/실습
Sprint 실행 및 Task 구현	통합 아키텍처 + Sprint 현황판 + 팀별 데모 시연 시나리오	3h	종합실습
Sprint Review & Retrospective	Review 진행가이드 + 계획 대비 실행 갭 분석 + 전체 Retro 시나리오	1h	개념실습

MSA (Microservice Architecture) 개요

- 모놀리식(Monolithic) 아키텍처와의 차이 - 단일 배포 단위 vs 독립 배포 가능한 서비스 단위
- 서비스 분리 기준 - 도메인 경계(Bounded Context), 데이터 소유권, 변경 빈도 등
- Day1에서 설계한 User Story가 어떤 MSA 컴포넌트(인증서버/Eureka/Gateway/Kafka)에 매핑되는지 매칭
- 다음 슬라이드에서 Agile 기반 Sprint 분할 로드맵과 SOLID 리팩토링 과정을 상세히 다룹니다

MSA는 정말 필요한가?

마이크로서비스 아키텍처로 달성해야 하는 Biz. 요구사항이 명확히 정의되어야 함

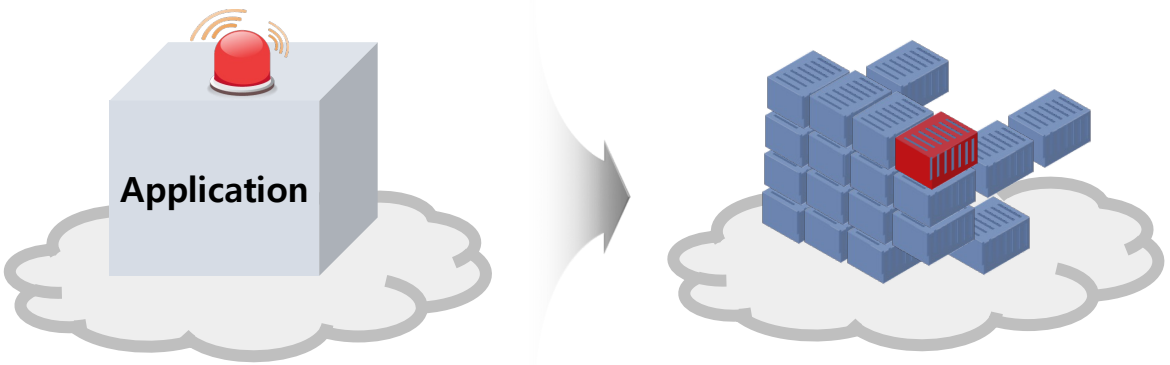


“작고, 독립적인”

DT Platform으로써 XaaS (PaaS)

Microservice?

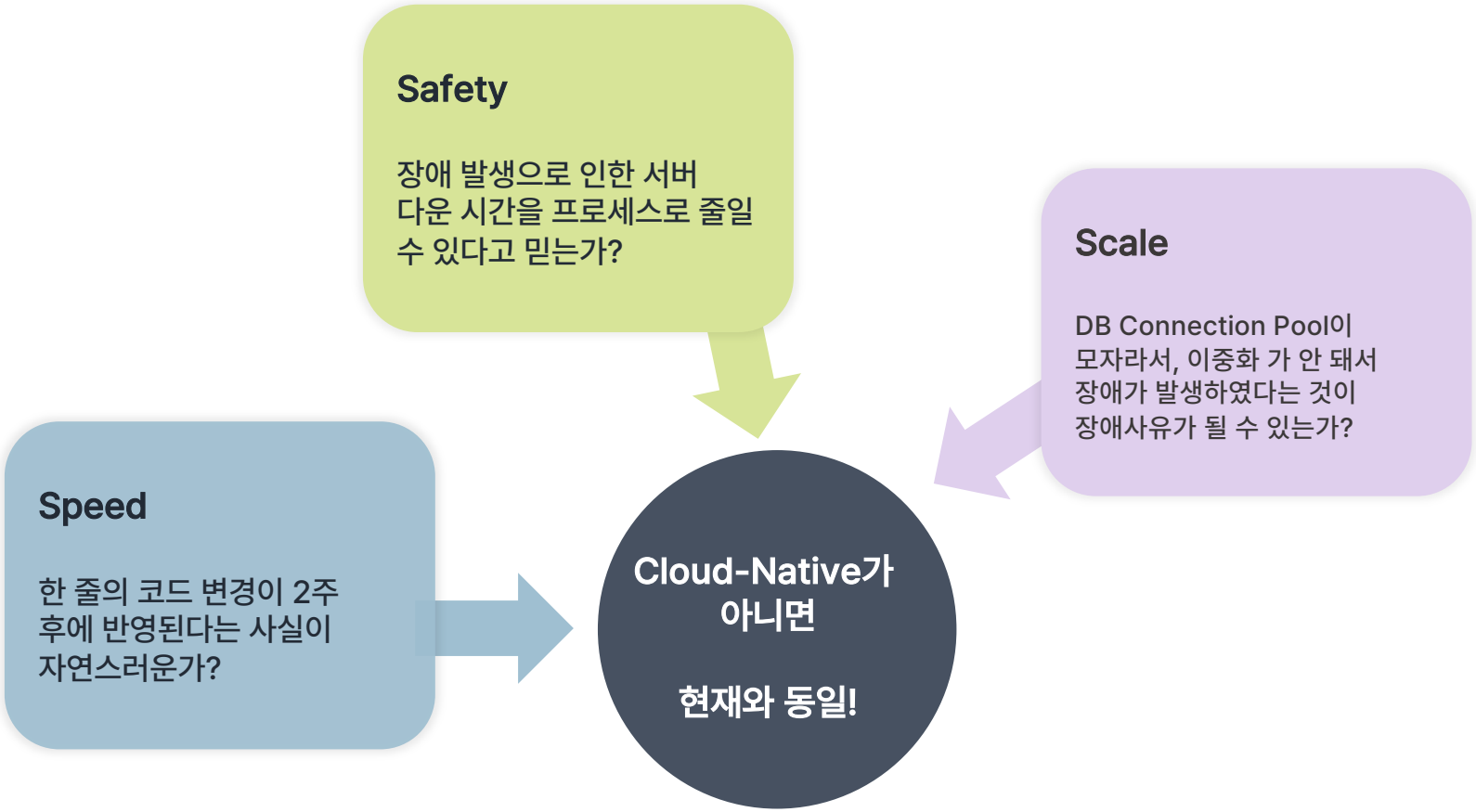
Application을 작은 단위로 쪼개 개발 : MicroServiceArchitecture(MSA)



- **기존 방식 On Premise :** 인프라, 서비스, 보고 생성을 모두 직접 관리
- **IaaS :** Infrastructure as a Service (인프라는 제공, 서비스 아키텍처 직접 관리)
- **PaaS :** Platform as a Service (인프라/서비스 아키텍처 관리 제공, 앱만 관리)
- **XaaS :** Anything as a Service (보고/결과만 받음)

DT Platform으로써 XaaS (PaaS)

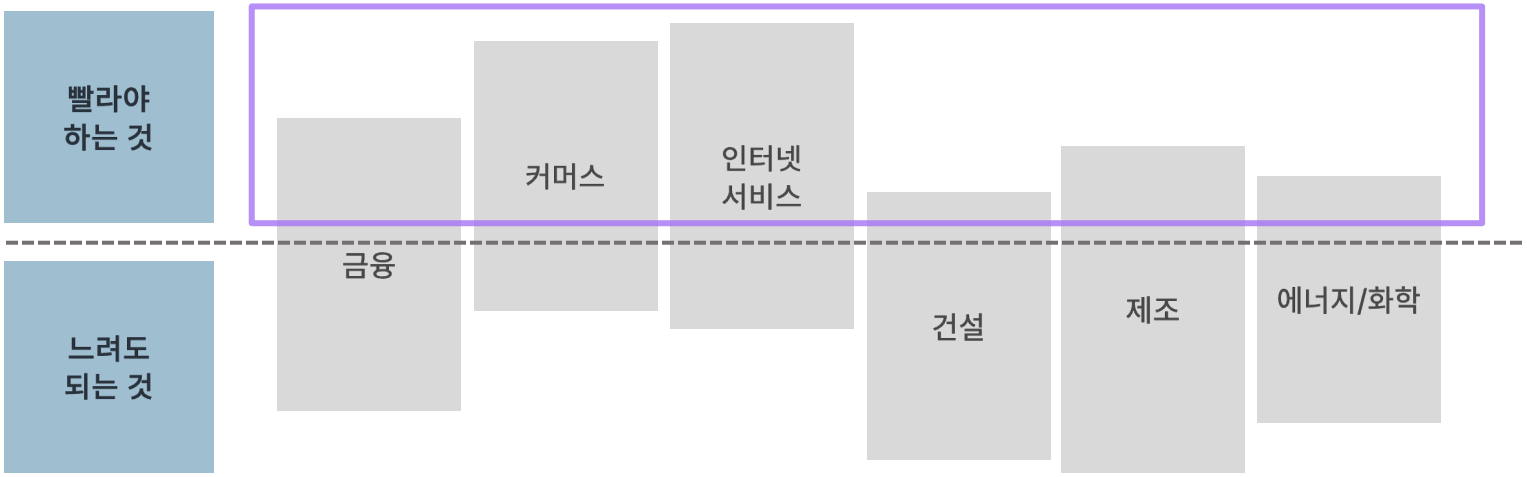
MSA의 효과?



DT Platform으로써 XaaS (PaaS)

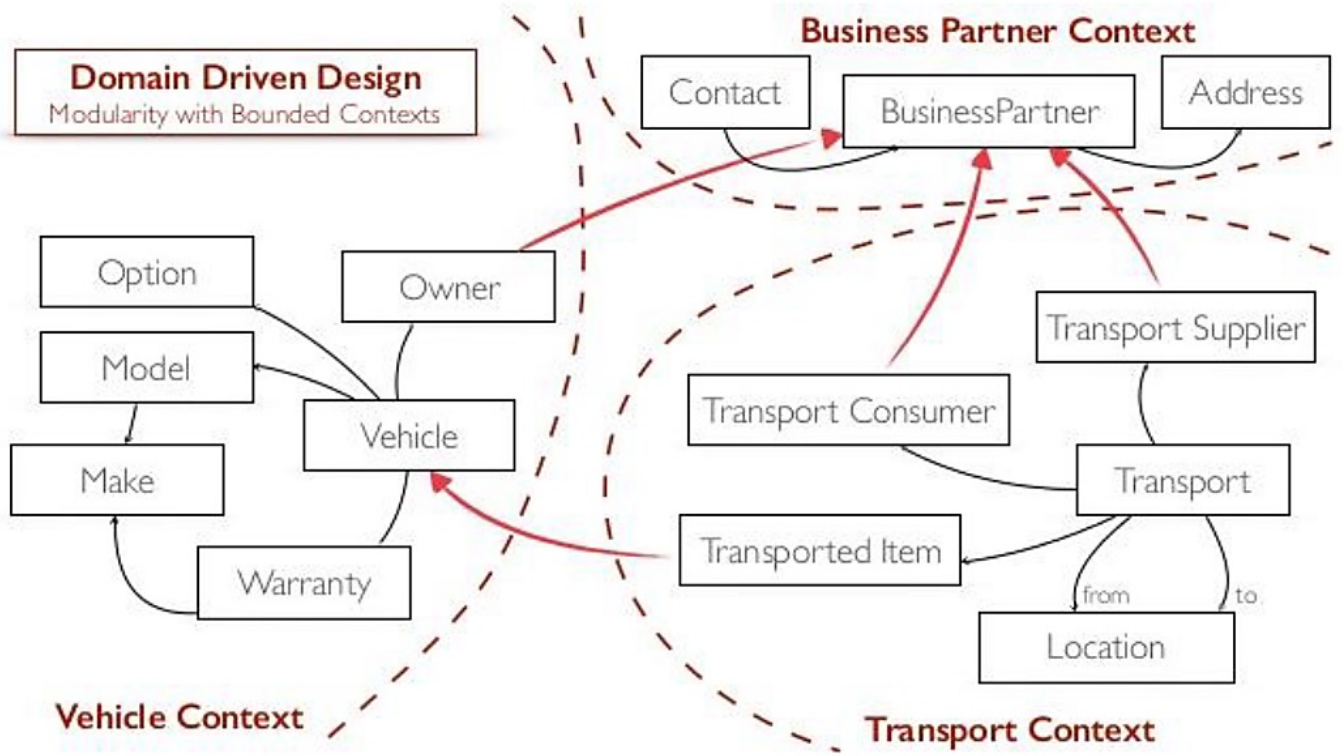
MSA 적합한 영역?

기존 IT서비스 대상 중 적합한 영역 vs 완전히 새로운 영역



DT Platform으로써 XaaS (PaaS)

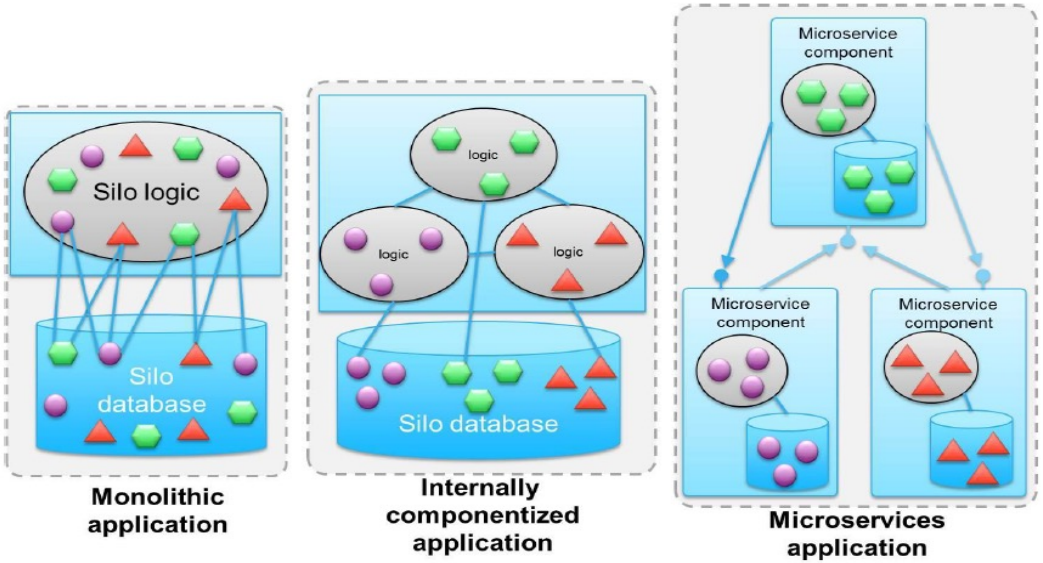
Bounded Context Map

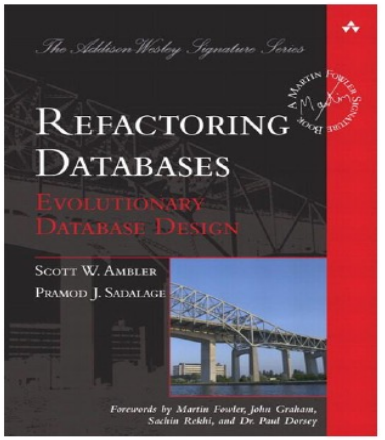


DT Platform으로써 XaaS (PaaS)

DB는 어떻게?

Application 모듈화 → API만으로 Communication → Database 분리





DT Platform으로써 XaaS (PaaS)

유형별 상세 비교

특징	Cloud Application 유형		
	Cloud Ready	Cloud Friendly	Cloud Native
전환 유형	Replatforming	Refactoring	ReAchitecturing
전환 이점	PaaS 플랫폼을 사용하는 이점	자동으로 스케일링 가능 설정자동화절차 체계화 해서 새로운 환경이나 개발자가 프로젝트에 참여하는데 시간과 비용 최소화	새로운 요구사항이나, 장애 등에 빠르게 대응 할 수 있음
전환 고려사항	PaaS에 배포하기 위해서 수정해야 되는 내용 도출	Scalabliity , 설정 자동화, 코드와 환경에 대한 내용 분리 , CI/CD	Microservice
기술 요건	PaaS 플랫폼 특징 이해	12factor , 배포 파이프라인	Design for failure, API first Design , Event driven Design
전환 목적	진화된 Cloud Applcation으로 발전하기 위한 단계로 활용	현재 아키텍처 수준에서 클라우드 효과를 최대로 보기위한 전환	변화나 수정에 유연한 구조의 애플리케이션으로 전환

- Cloud Application 유형분리는 절대적인 기준이 아니며, Cloud에 최적화되어 최대의 효과를 얻을 수 있는 애플리케이션으로 전환을 한번에 이루기 어렵기 때문에 단계적으로 진화 가능한 수준이라는 것을 보여주기 위한 분리이다.
- 실제로 애플리케이션 전환 시 기존시스템의 특성에 따라 최대의 효과를 볼 수 있는 전환 수준을 도출 해야 한다 .
  (예> Cloud ready수준에 scalability만 확보하면 최소의 노력으로 최대의 효과를 볼 수 있다고 분석되면 Cloud Ready단계의 고려사항과 Cloud Friendly 단계의 고려사항 중 scalability 확보를 위한 기술 까지만 적용.)
- Cloud Ready < Cloud Friendly < Cloud Native
  (Cloud Native 애플리케이션 으로 전환 설계 시 Cloud Ready , Cloud Friendly 의 특징을 모두 고려해야 함)

DT Platform으로써 XaaS (PaaS)

Replatforming?

[예시] Cloud Ready 상태로 만들기 위한 고려사항

JDK	
고려 사항	**설명**
JDK 버전	빌드팩에서 지원되는 JDK 버전을 확인하고 업그레이드 해야함

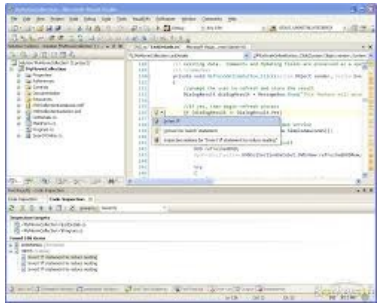
Library	
고려 사항	**설명**
Spring Framework 버전	Spring FWK 사용 시 빌드팩에서 지원되는 버전을 확인하고 업그레이드 해야 함
WAS 및 JAVA에 대한 Dependency	WAS, JDK 변경 및 업그레이드에 따른 라이브러리 오류 발생 시 최신 library로 대체하거나 불필요한 library는 삭제하고 필요한 경우 소스를 수정함 ex) JDK 버전 업그레이드 시 JDBC library 오류가 발생할 경우 JDBC도 최신 library로 변경해야 함
3rd party library	애플리케이션에서 사용하고 있는 3rd party library가 PaaS에서 구동 가능한지 확인 필요에 따라 PaaS 환경에 맞는 라이브러리로 교체 (ex. 설치형 라이브러리)
기타	사용하고 있는 3rd party library의 라이선스가 Cloud 환경에서 적합한지 확인

DT Platform으로써 XaaS (PaaS)

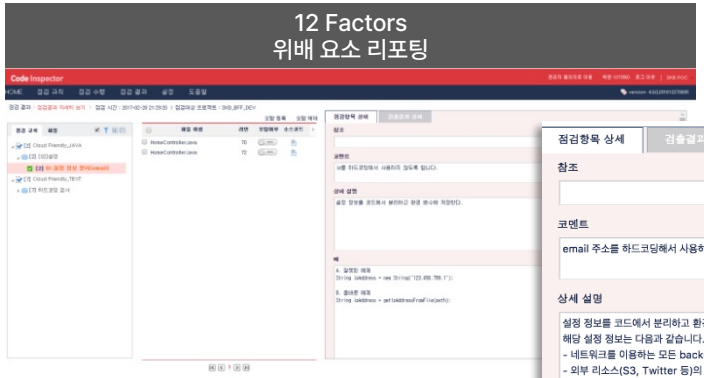
Refactoring?

[예시] Cloud Friendly 상태로 만들기 위한 12 Factors 리팩토링

12 Factors 위배 요소
Error 마킹

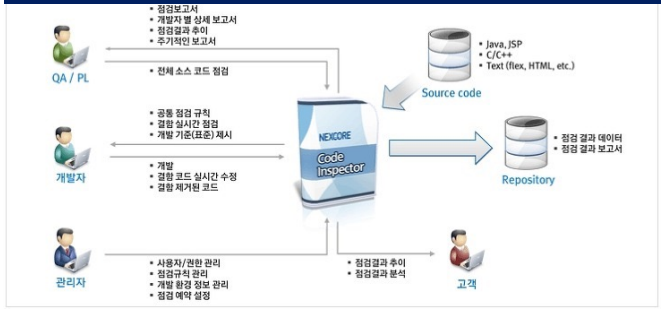


12 Factors
위배 요소 리포팅





Code Inspector for 12 factors
on DevOps





12 Factors
위배 요소 검출 Sample

DT Platform으로써 XaaS (PaaS)

12 factors?

Cloud Application 구축을 위한 표준 설계 원칙 개발 방법 및 개발/운영 환경 포함

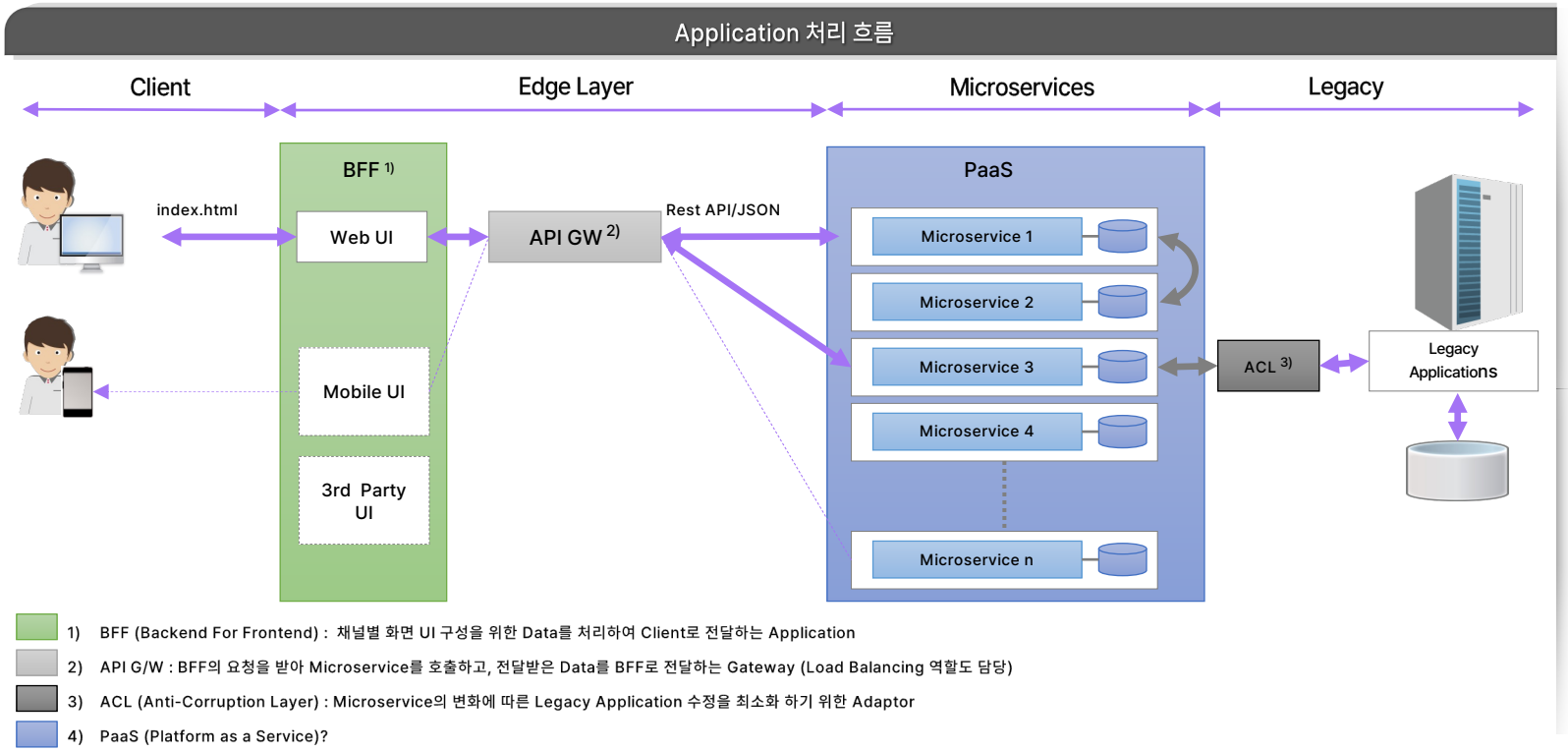
PaaS와 DevOps만으로 4~7 Factors 달성

12 Factors		달성 방법	PaaS	DevOps	Refactor
1. 동일한 Repo	개발/운영을 하나의 저장소로 관리	DevOps Pipeline		✓	
2. 의존성 설정	소스가 아닌 설정으로 의존성 관리	Maven			✓
3. 환경변수	소스가 아닌 환경변수로 관리	ENV, Spring Config			✓
4. 서비스바인딩	DB, 연계는 서비스바인딩으로 간주	Backing Services	✓		
5. 릴리스 분리	빌드/릴리스/실행의 완전한 분리	DevOps Pipeline		✓	
6. 무상태	무상태 처리를 기본으로 함	Stateless/Shared Nothing			✓
7. Self-Contained	컨테이너가 아닌 Appl. 자체가 URL	Menifest & Container	✓		
8. 동시성	스케일 아웃을 위한 프로세스 모델	Router, Microservices			✓
9. 빠른 Reboot	빠른 부팅과 Graceful 셧다운	Cloud Foundry Container	✓		
10. 실행환경일치	개발계/검증계/운영계 환경 일치	DevOps Pipeline		✓	
11. 중앙 로깅	플랫폼에서 이벤트 로그를 수집	Metric & Logging	✓		
12. JOB 관리	DB변경과 같은 JOB을 소스에 포함	Infrastructure as a Code			✓

DT Platform으로써 XaaS (PaaS)

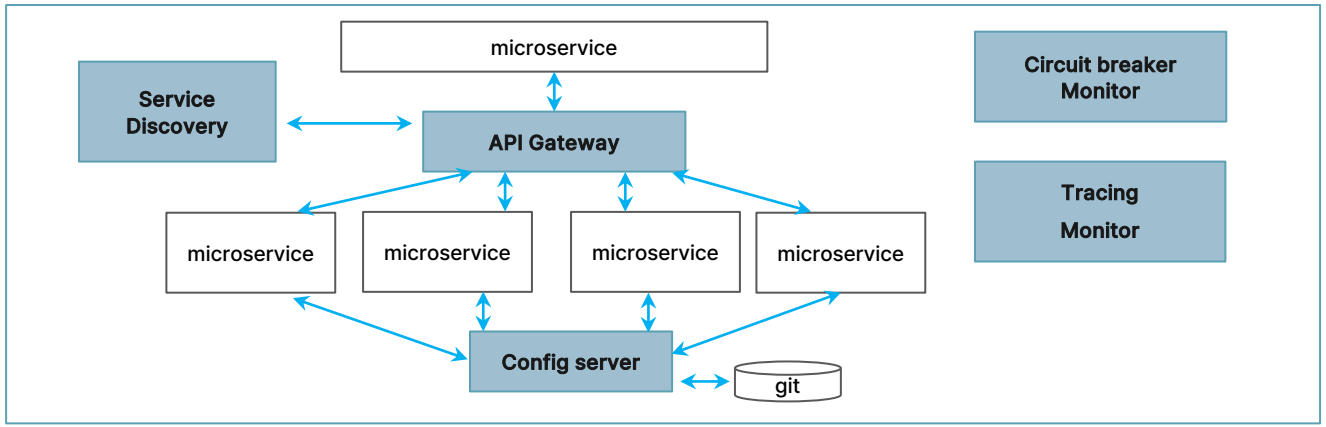
ReArchitecture?

[예시] Cloud Native 상태로 만들기 위한 Microservice 아키텍처 설계



Component for MSA Service

구성요소



구성요소	설명
API Gateway	API 단일 진입 포인트 , 인증 권한을 부여하거나, 프로토콜 변경 등의 단일 포인트가 된다.
Service Discovery	Microservice들간 동적 참조를 위해서 각 서비스 의 네트워크를 등록하는 역할 서비스 명으로 서비스간 통신이 가능하도록 해준다.
Config Server	마이크로 서비스로 구성한 여러 개의 설정 정보를 한곳에 모아서 관리 포인트 일원화 (git 저장소) 설정 정보가 변경되었다고 하더라도 앱을 다시 재 빌드/ 배포 하지 않고, git 저장소에 변경된 정보를 push 하면 변경된 정보 사용 가능
Tracing Monitor	여러 개의 마이크로 서비스로 구성된 전체 애플리케이션을 모니터링 서비스의 타이밍 데이타를 모아서 잠재적인 문제를 해결하는 자료로 사용 가능하고, 서비스들끼리의 요청의 종속성을 확인 할 수 있음.
Circuit Breaker & monitor	마이크로 서비스 아키텍처 구조상 통신이 많아지는데 한 서비스의 장애가 다른 서비스에 전이를 막기 위해 오류가 있는 서비스에 대해서 서킷 브레이커를 발동시켜 사용 불가상태로 전환, 연쇄적인 오류를 방지하기 위한 목적으로 사용한다.

모놀리식 인증/로그인 vs SOLID 적용 구조

MSA 개요 · 1/2 · 실전 적용

Before - Pure Java 단일 클래스

- LoginService 클래스 하나가 입력 검증, 비밀번호 확인, 세션 생성, 로그 기록을 모두 담당
- 책임이 얽혀 있어 인증 로직만 따로 떼어 재사용하거나 다른 서비스로 옮기기 어려움
- 테스트 시 전체 클래스를 실행해야 하며, 일부 로직만 검증하기 힘듦

After - SOLID 적용 후 구조

SRP CredentialValidator / SessionManager / AuthLogger로 책임 분리

OCP 인증 방식(비밀번호/OAuth) 추가 시 기존 코드 수정 없이 확장

DIP UserRepository 인터페이스에 의존, 구현체는 주입받아 교체 가능

→ *분리된 CredentialValidator/SessionManager가 각각 인증서버의 독립 컴포넌트로 이관됩니다.*

SOLID 원칙 적용 예시 (Reminder-Java)

약자	원칙(영문)	원칙(한글)	핵심 내용	Java SpringBoot 예시
S (SRP)	**S**ingle **R**esponsibility **P**rinciple	**단일 책임 원칙**	하나의 클래스는 하나의 책임만 가져야 한다.	Controller, Service, Repository 등 계층 분리
O (OCP)	**O**pen-**C**losed **P**rinciple	**개방-폐쇄 원칙**	기존 코드는 수정하지 않고 기능을 확장할 수 있어야 한다.	인터페이스 기반 구현체 추가, 전략 패턴 활용
L (LSP)	**L**iskov **S**ubstitution **P**rinciple	**리스코프 치환 원칙**	부모 타입 대신 자식 객체를 사용해도 동일하게 동작해야 한다.	인터페이스 구현체 교체 가능
I (ISP)	**I**nterface **S**egregation **P**rinciple	**인터페이스 분리 원칙**	필요한 기능만 가진 작은 인터페이스로 분리한다.	역할별 인터페이스 설계
D (DIP)	**D**ependency **I**nversion **P**rinciple	**의존성 역전 원칙**	구현 클래스가 아닌 추상화(인터페이스)에 의존한다.	Spring의 DI와 IoC의 핵심 원칙

실행 시나리오 - 인증/로그인 컴포넌트 분리 매핑

MSA 개요 · 2/2 · 실행 시나리오

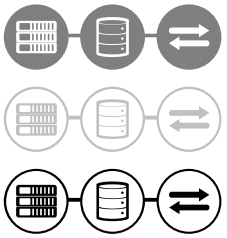
기존 모놀리스 컴포넌트	SOLID 적용 후 역할	이관될 MSA 서비스
CredentialValidator	회원 자격 검증(SRP 적용)	인증서버 (Auth Server) 내부 로직
SessionManager / TokenIssuer	세션 관리 → JWT/OAuth 토큰 발급으로 전환	인증서버 (Auth Server) OAuth 모듈
LoginController	요청 수신 및 응답 반환	API Gateway 진입점 + 인증서버 REST API
AuthLogger	인증 이력 기록	이벤트 발행 후 Kafka로 비동기 전달 (Sprint 3)

이 매핑표가 Sprint 1의 Product Backlog 항목(= User Story)이 됩니다. 다음 슬라이드에서 Task로 분할합니다.

인증서버 & API Gateway

인증서버 & GATEWAY · 1/4

- 인증(Authentication)/인가(Authorization) 흐름 - OAuth 2.0 및 JWT 토큰 발급·검증
- API Gateway의 역할 - 모든 요청의 단일 진입점, 라우팅 및 인증 필터 처리
- 제공되는 기반 코드 구조 설명 (인증서버 프로젝트 / Gateway 프로젝트)
- 다음 슬라이드에서 Sprint 1 Planning부터 실제 구현 프로세스까지 다룹니다.



Sprint 1 Planning - 인증서버(OAuth) 구축 Task 분할

인증서버 & GATEWAY · 2/4

Sprint 목표 : Pure Java 인증/로그인을 SOLID 적용 후 Spring Boot 기반 OAuth 인증서버로 전환한다

Task	세부 내용	Point
기존 로그인 로직 분석	CredentialValidator/SessionManager 등 책임 단위 식별	3
SOLID 리팩토링	SRP/OCP/DIP 적용하여 인터페이스·클래스 분리	5
Spring Boot 프로젝트 구조화	분리된 클래스를 Spring Boot 프로젝트로 이관	3
OAuth 2.0 Authorization Server 설정	Spring Authorization Server 설정 및 토큰 발급 엔드포인트 구현	5
Gateway 라우팅 설정	인증서버로 향하는 라우팅 규칙 및 인증 필터 구성	3

SOLID → Spring Boot OAuth 인증서버 구현 프로세스

인증서버 & GATEWAY · 3/4

1. **기존 Pure Java 로그인 로직 분석**
   LoginService 내부에서 검증/세션/로깅 책임이 어떻게 얽혀 있는지 식별

2. **인터페이스 분리 (SRP/DIP 적용)**
   CredentialValidator, TokenIssuer 등 인터페이스로 책임을 분리하고 구현체는 주입

3. **Spring Boot 프로젝트로 이관**
   분리된 클래스를 @Service/@Component로 등록, 기존 로직은 최대한 재사용

4. **OAuth 2.0 Authorization Server 구성**
   spring-security-oauth2-authorization-server로 토큰 발급 엔드포인트 구현

5. **Gateway 인증 필터 연동**
   Gateway에서 발급된 JWT를 검증하는 필터를 추가해 인증서버와 연결

실행 시나리오 - Gateway 라우팅 & 인증 필터 데모

인증서버 & GATEWAY · 4/4

수강신청 시스템 로그인 요청 흐름

학생 로그인 요청 > API Gateway 수신 > 인증서버로 라우팅 > OAuth 토큰 발급 > **Gateway 필터 검증**

데모 진행 대본

- ① 학생이 Postman(또는 프론트)으로 /api/login에 학번/비밀번호를 전송
- ② Gateway가 요청을 인증서버로 라우팅 (Eureka에서 위치 조회, Sprint 2에서 연동)
- ③ 인증서버가 CredentialValidator로 자격을 검증 후 OAuth 토큰(JWT)을 발급
- ④ 이후 모든 요청은 Gateway의 인증 필터가 토큰을 검증한 뒤 대상 서비스로 전달

서비스 디스커버리(Eureka) & 서비스 간 통신

EUREKA & 서비스 통신 · 1/4

- Eureka의 서비스 등록(Register)/조회(Discovery) 원리
- 동기(REST) 통신과 비동기(Kafka) 통신의 차이 및 각각의 적용 상황
- 서비스 간 통신 시 장애 대응(Fallback 등) 개념 소개
- 다음 슬라이드에서 Sprint 2 Planning부터 실제 연동 과정까지 다룹니다



Sprint 2 Planning - Eureka 연동 Task 분할

EUREKA & 서비스 통신 · 2/4

Sprint 목표 : 인증서버·Gateway·수강신청 서비스가 서로를 자동으로 찾아 통신하게 한다.

Task	세부 내용	Point
Eureka Server 구축	Spring Cloud Netflix Eureka Server 프로젝트 생성 및 실행	3
각 서비스에 Eureka Client 등록	인증서버·Gateway·수강신청 서비스에 @EnableEurekaClient 적용	3
Gateway 동적 라우팅 설정	서비스명 기반 라우팅으로 전환(고정 IP 제거)	3
서비스 간 REST 호출 구현	수강신청 서비스가 인증서버 사용자 정보를 조회하는 REST 클라이언트 구현	5

Eureka 등록/조회 실전 연동 프로세스

EUREKA & 서비스 통신 · 3/4

1. **Eureka Server 실행**
   spring-cloud-starter-netflix-eureka-server 의존성 추가 후 기본 포트(8761)로 기동

2. **각 서비스에 Client 설정 추가**
   application.yml에 eureka.client.service-url 지정, @EnableEurekaClient 어노테이션 부여

3. **서비스 등록 확인**
   Eureka 대시보드에서 인증서버·Gateway·수강신청 서비스가 모두 등록되었는지 확인

4. **Gateway 라우팅을 서비스명 기반으로 전환**
   lb://AUTH-SERVICE 형태로 라우팅 규칙 수정

5. **서비스 간 호출 테스트**
   수강신청 서비스에서 인증서버를 이름으로 호출해 정상 응답 확인

실행 시나리오 - 수강신청 서비스의 인증서버 탐색 흐름

EUREKA & 서비스 통신 · 4/4

수강신청 요청 수신 › Eureka에 AUTH-SERVICE 위치 조회 › 인증서버로 REST 호출 › 사용자 정보 확인 › **수강신청 처리**

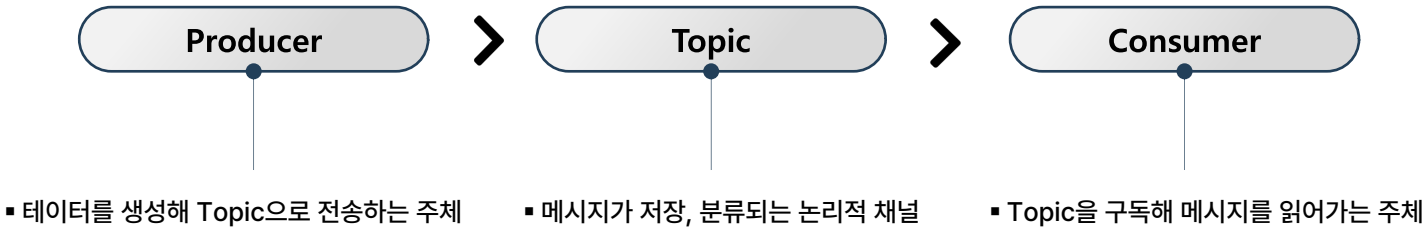
데모 진행 대본

- ① 학생이 로그인 후 발급받은 토큰으로 /api/courses/apply 요청
- ② 수강신청 서비스가 Eureka에게 "AUTH-SERVICE 어디 있어?"라고 조회
- ③ Eureka가 인증서버의 현재 위치(IP:Port)를 응답
- ④ 수강신청 서비스가 해당 위치로 직접 REST 호출하여 사용자 유효성 확인 후 신청 처리

Kafka 기본

KAFKA 기본 · 1/4 · 실전 적용

- Producer / Consumer / Topic 핵심 개념
- MSA에서 이벤트 기반(비동기) 통신이 필요한 이유 - 서비스 간 느슨한 결합
- 제공되는 기반 코드의 Kafka 연동 구조 설명
- 다음 슬라이드에서 Sprint 3 Planning부터 수강신청 이벤트 데모까지 다룹니다.



Sprint 3 Planning - Kafka 연동 Task 분할

KAFKA 기본 · 2/4 · 실전 적용

Sprint 목표 : 수강신청 완료 시 이벤트를 발행하고, 다른 서비스가 이를 비동기로 구독한다.

Task	세부 내용	Point
Kafka 브로커 기동 및 Topic 생성	course-applied Topic 생성, Partition/Replication 기본값 설정	2
Producer 구현	수강신청 서비스에 KafkaTemplate으로 이벤트 발행 로직 작성	3
Consumer 구현	알림 서비스(또는 로그 서비스)에서 @KafkaListener로 이벤트 구독	3
통합 테스트	수강신청 → 이벤트 발행 → 구독 서비스 로그 확인까지 End-to-End 검증	3

Kafka Producer/Consumer 구현 프로세스

KAFKA 기본 · 3/4 · 실전 적용

1. **Topic 설계**
   "course-applied" 등 이벤트 성격이 드러나는 이름으로 Topic 이름 결정

2. **Producer 작성**
   수강신청 처리 로직 마지막에 KafkaTemplate.send()로 이벤트 발행

3. **이벤트 메시지 설계**
   학번·과목코드·신청시각을 포함한 최소한의 JSON 페이로드로 구성

4. **Consumer 작성**
   @KafkaListener(topics="course-applied")로 구독 서비스에서 이벤트 수신

5. **장애 대응 고려**
   Consumer 처리 실패 시 재시도 또는 Dead Letter Topic 개념 소개

실행 시나리오 - 수강신청 완료 이벤트 발행/구독 데모

KAFKA 기본 · 4/4 · 실전 적용

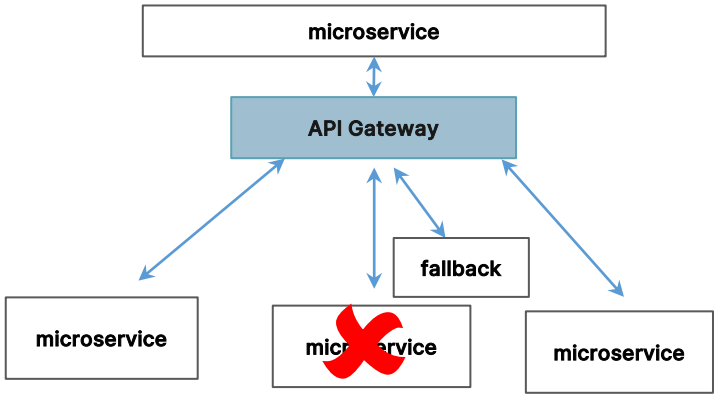
수강신청 완료 → course-applied 이벤트 발행 → Kafka Topic 적재 → 알림 서비스 구독 → 완료 알림 로그 출력

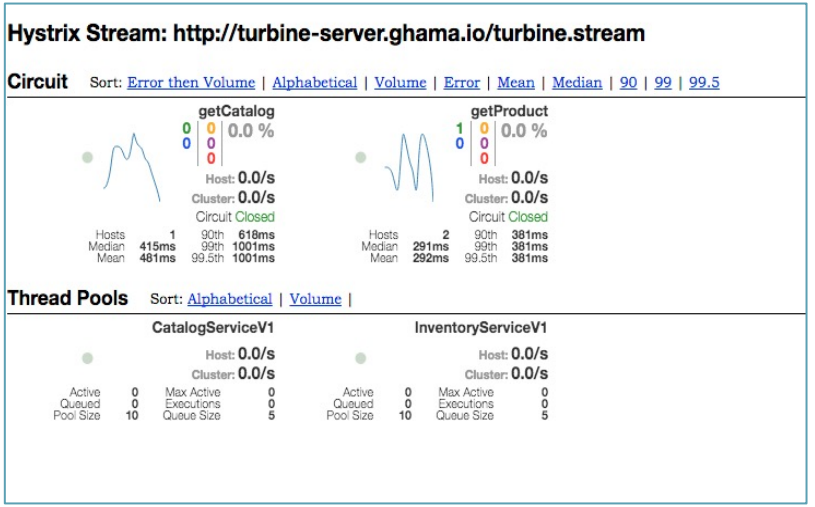
데모 진행 대본

- ① 학생이 원하는 과목에 수강신청을 완료
- ② 수강신청 서비스가 course-applied Topic으로 이벤트(학번/과목코드/시각)를 발행
- ③ 알림 서비스가 해당 Topic을 구독하고 있다가 이벤트를 즉시 수신
- ④ 알림 서비스 콘솔에 "OOO님 OOO 과목 신청 완료" 로그가 실시간으로 출력됨을 시연

서비스간 통신

Circuit Breaker





역할

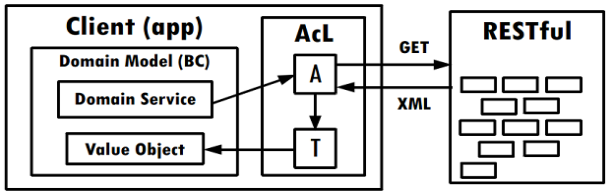
실패에 대한 설계

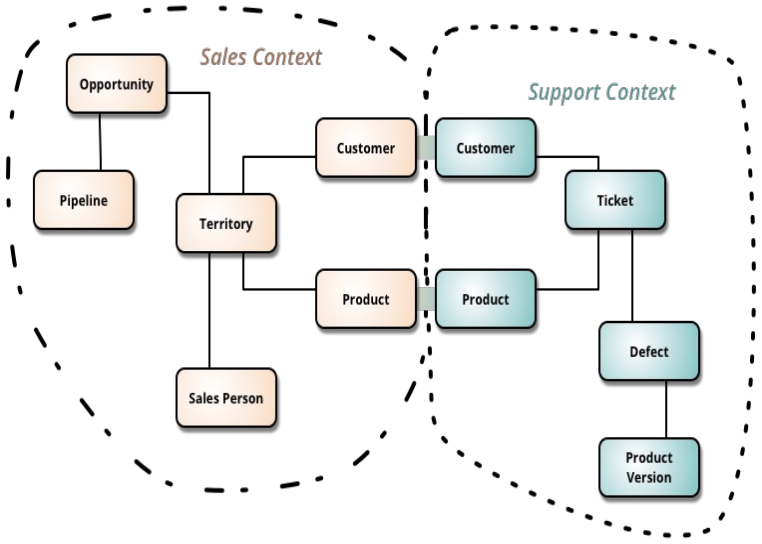
서비스간 의존성이 발생하는 접근 포인트에 설계해서 장애 전파를 막고 fallback 함수를 지원한다.

현재 Circuit breaker 의 상태를 대시보드를 통해서 모니터링 한다.

서비스간 통신

Anti-corruption Layer



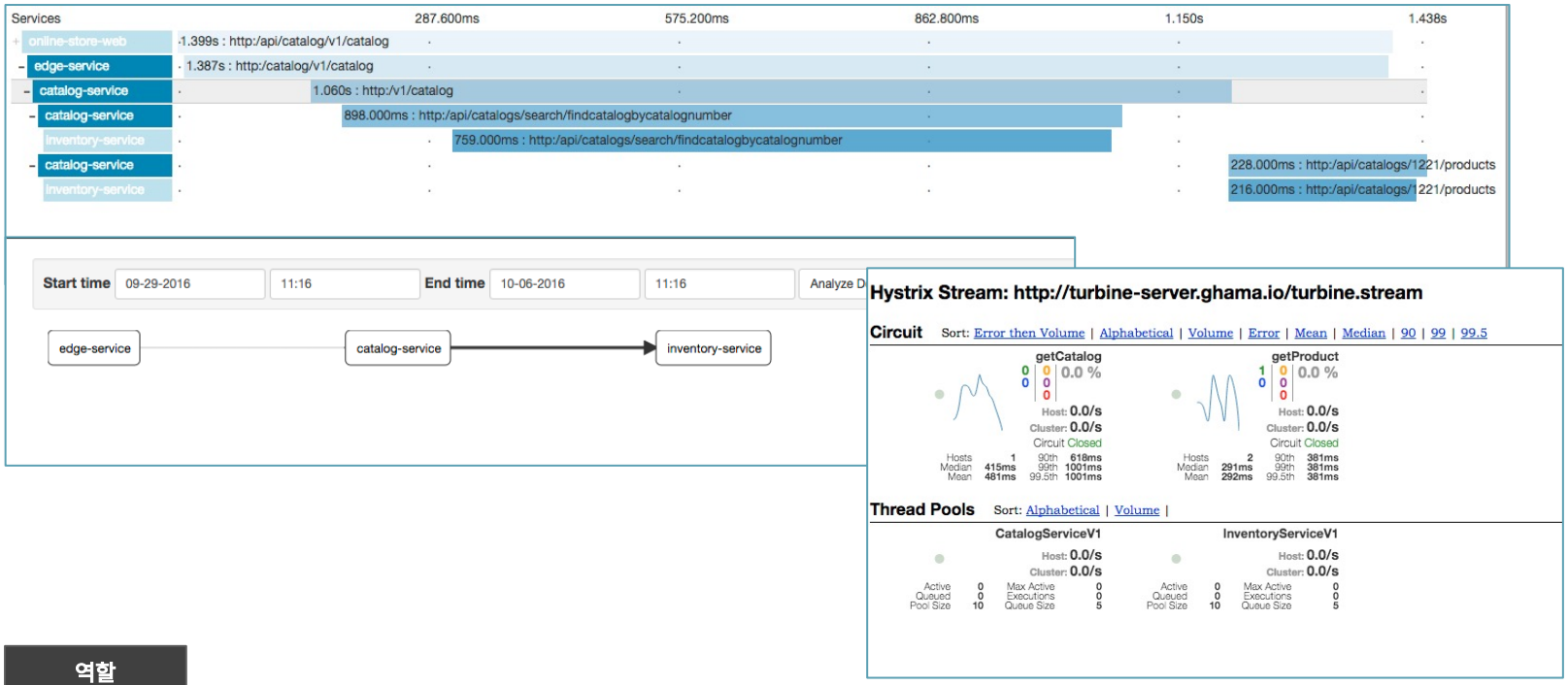


역할

서비스간에는 API만으로 통신하고, 서로 모델을 공유 하지 않는다.
필요한 경우 API를 호출한 서비스에서 Anti corrution layer를 두고 API 결과를 이 영역에서 가공한다.
마이크로서비스는 각각의 서비스가 다른 서비스와 영향 없이 독립적으로 개발하는 개발용이성이 중요한 포인트이므로 서비스간 모델을 서로 공유하는 형태가 아니라 API의 결과를 가지고 자기 서비스에서 가공하여 사용하는 방식으로 개발한다.
(서비스별 중복 코드 발생 O, 중복 데이터 발생 O)

서비스간 통신

Monitoring



역할

마이크로서비스 여러 개가 하나의 단일 애플리케이션으로 구성되므로 서비스간의 연계에서 오는 복잡도가 증가함에 따라서
단일 서비스는 물론 여러 개의 마이크로 서비스의 연계관계에 대한 모니터링이 필요함

MSA에서 Data처리

MSA 유의사항 (Case Study)

데이터 처리중심은 APP'서버일까? DB서버인가?

테이블 읽기

EMPNO	SAL	CODE	
7782	2450	3	12:00 기준 select
7839	5000	2	
7934	100	5	
7369	800	1	
7566	2975	1	
7788	3000	5	
7876	1100	7	
7902	3000	6	
7499	1600	2	
7521	1250	2	7521번 1250 -> 5000 변경
7654	1250	3	
7698	2850	5	
7844	1500	6	
7900	950	7	변경값이 포함되는가?

[SQL Trace 결과]

Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	
Parse	1	0.010	0.004	0	0	0	0
Execute	1	0.050	0.042	0	0	0	0
Fetch	3	0.620	0.638	5	6683	0	20
Total	5	0.680	0.684	5	6683	0	20

[AutoTrace 결과]

Statistics

```
----------------------------------------------------------
      5  recursive calls
     30  db block gets     -- current mode
     31  consistent gets   -- consistent mode
      0  physical reads
      0  redo size
    680  bytes sent via SQL*Net to client
    625  bytes received via SQL*Net from client
      3  SQL*Net roundtrips to/from client
      1  sorts (memory)
      0  sorts (disk)
     14  rows processed
```

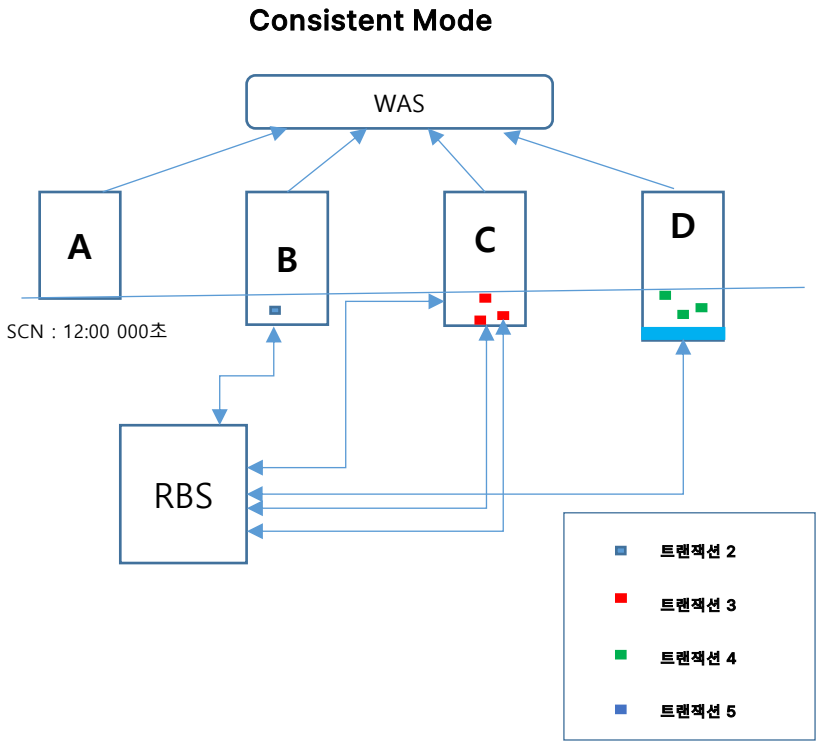
MSA에서 Data처리

DBMS에서 데이터를 처리할 때 SQL 조인을 사용하는 경우와 조인을 사용하지 않고 WAS, 어플리케이션서버로 가져가서 처리하는 경우를 살펴보자. 조인을 사용하는 경우는 다음과 같다. 트랜잭션이 시작하는 순간부터 끝날때까지 데이터 일관성과 다른 트랜잭션의 영향을 받지 않도록 데이터를 자동 처리한다. 오라클은 타임스탬프 역할을 하는 SCN을 이용하여 SQL이 시작하는 시점을 기준으로 다른 테이블을 조인하여도 타 트랜잭션에 의해 Commit(변경)된 데이터를 읽지 않도록 데이터를 일치시켜준다(Consistent Mode). 즉 타 트랜잭션이 변경한 데이터에 영향을 받지 않도록 DB Isolation Level을 지켜주고 있다. WAS로 데이터를 가져갈 경우 테이블의 데이터가 변경되면 잘못된 값을 WAS에서 처리하게 될 수 있는데 이는 타 트랜잭션의 데이터 변경 영향을 받게 되는 것으로 트랜잭션 격리화 수준에 위배된다.

1. 12시 00분 00초에 sql 조회 시작
2. 타 트랜잭션 2 ~ 5가 12시 이후로 데이터를 수정하고 commit함.
3. Sql 조인은 시작한 시점을 기준으로 SCN을 이용하여 조회하고 타 트랜잭션이 commit한 수정한 값들의 영향을 받지않음. => 트랜잭션 격리화 시작시점의 SCN과 비교하여 값이 차이가 있으면 RBS(롤백세그먼트)를 이용해서 시작시점의 값을 찾아내서 데이터 시점을 맞춤
4. 즉 12시 00분 00초시점에 시작된 SQL은 시작시점에 전 테이블의 데이터를 동기화하여 조회한다.
5. DBMS에서 자동으로 제공하는 트랜잭션 격리화 수준 을 이용하면 정확한 값을 쉽게 얻을수있다.

조인 쿼리

```
select A.번호,  B.주소, C.점수, D.주소
from  A, B, C, D
where A.번호 = B.번호
   and B.번호 = C.번호
   and C.번호 = D.번호;
```

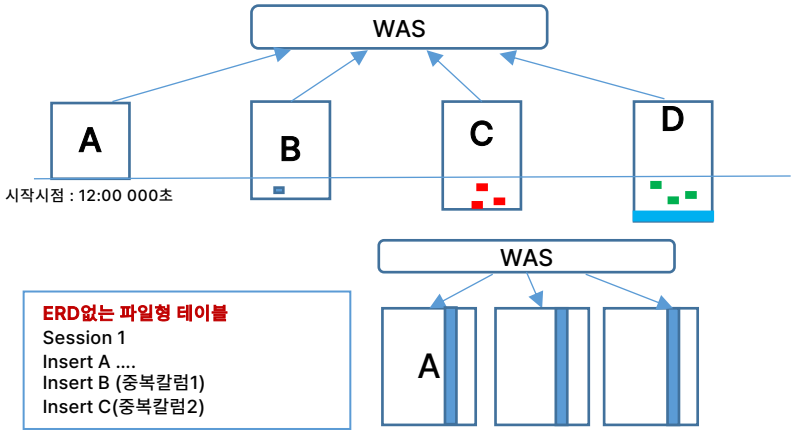


MSA에서 Data처리

WAS나 어플리케이션 서버로 가져가서 처리하는 경우를 살펴보면 A테이블을 조회하는 SQL이 12시 00분 00초에 시작되었다고 가정하면 데이터를 WAS로 가져오는 2번째 및 3, 4번째 테이블 등의 데이터는 A테이블이 조회하는 시점보다 늦어 그 데이터에는 다른 트랜잭션이 COMMIT한 수정값들이 포함되어 있을 수 있다.(Dirty Read, Phantom Read등도 발생 가능함) 그러므로 WAS로 데이터를 가져와서 처리하는 경우는 다른 트랜잭션들의 수정값들을 반영하므로 트랜잭션 격리화 수준에 위배가 된다. 이를 방지 하려면 A테이블의 데이터를 가져오기 전에 전테이블에 LOCK를 사용하여 DML을 방지하여야 한다. MSA에서 조인을 하지 않기 위해 테이블을 파일형처럼 사용하는 경우가 있는데 이도 또한 격리화 수준에 위배될 수 밖에 없고 중복되는 칼럼들의 데이터 동기화를 위해 데이터를 2번 이상 동일 처리하는 방법을 사용하는 경우가 발생하는데 이는 대용량 DB에는 심각한 성능저하를 발생시킬 수 있다.

1. A테이블에서 12시에 데이터를 조회하여 WAS로 가져온다.
2. B테이블에서 해당 데이터를 가져오나 A테이블의 조회시점보다 늦기때문에 12시00분00초이후의 수정된 COMMIT정보를 가져올 수 있다.
3. C, D테이블에서 데이터를 각각 가져온다. 12시 이후에 수정된 COMMIT 값들을 가져올 수 있다.
4. 가져온 값들을 WAS에 처리하면 잘못된 값이 만들어질 수 있다.

가정) 조회 시 1시간 정도 소요되고 테이블이 10개 정도 조회하는 경우가 있다면 심각하게 데이터가 안 맞을 수 있다.



WAS중심 데이타처리, MSA, 루핑형 SQL

```
--Session 1
Lock table A;
Lock table B;
Lock table C;
Lock table D;
--데이터를 WAS 단으로 가져가서 처리
Commit;
```

ERD없는 파일형 테이블

```
Session 1
Insert A ....
Insert B (중복칼럼1)
Insert C(중복칼럼2)
```

Commit;

* **MSA** 적용 시에도, **ERD 작성하고 SQL은 조인형태 사용** 필요

Sprint 실행 및 Task 구현 개요

Sprint 실행 및 Task 구현 · 1/4

STEP 1

Task 구현

- Day1 Sprint Backlog의 각 Task를 팀별로 분담
- 제공된 기반 코드 위에서 실제 구현 (예: 인증 서버 API 구현)
- Kafka 이벤트 발행/구독 로직 작성

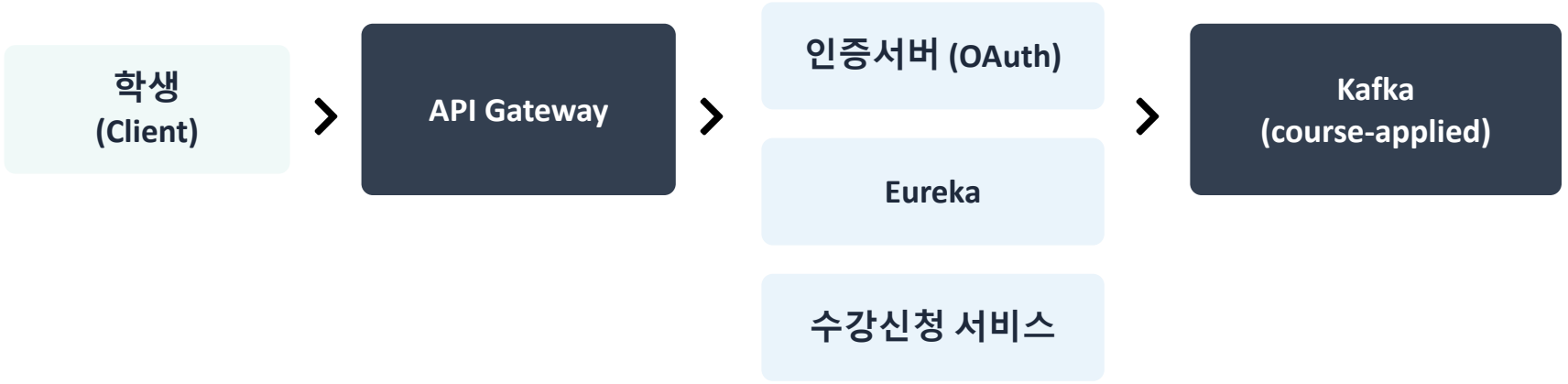
STEP 2

진행 상황 관리

- Sprint Board의 Task 상태를 실시간으로 업데이트
- Daily Scrum 형식으로 중간 진행상황 공유
- 장애물 발생 시 팀 내 즉시 공유 및 조율

수강신청 시스템 데모 아키텍처 통합

Sprint 실행 및 Task 구현 · 2/4



전체 데모 흐름

- ① 학생이 로그인 → Gateway → 인증서버(OAuth) → JWT 토큰 발급
- ② 학생이 과목 목록 조회 및 수강신청 → Gateway → Eureka로 수강신청 서비스 위치 탐색 → 서비스 호출
- ③ 수강신청 완료 → Kafka(course-applied) 이벤트 발행 → 알림 서비스가 구독하여 완료 알림 처리

Sprint별 Task 현황판 통합 뷰

Sprint 실행 및 Task 구현 · 3/4

Sprint	Task	상태
Sprint 1	인증서버 OAuth 구축	Done
Sprint 1	Gateway 인증 필터 연동	Done
Sprint 2	Eureka Server 구축 및 Client 등록	Done
Sprint 2	수강신청 서비스 ↔ 인증서버 REST 연동	In Progress
Sprint 3	Kafka Topic 및 Producer 구현	In Progress
Sprint 3	알림 서비스 Consumer 구현	To Do

Sprint 1~2는 완료되었고, Sprint 3(Kafka 연동)이 진행 중인 상태를 팀 전체가 공유합니다.

실행 시나리오 - 팀별 통합 데모 시연 순서

Sprint 실행 및 Task 구현 · 4/4

1. **로그인 데모 (3분)**
   OAuth 인증서버를 통해 학생 계정으로 로그인, 발급된 JWT 확인

2. **서비스 탐색 데모 (2분)**
   Eureka 대시보드에서 등록된 서비스 목록과 상태 확인

3. **수강신청 데모 (3분)**
   발급받은 토큰으로 과목 조회 후 수강신청 요청 실행

4. **이벤트 처리 데모 (2분)**
   Kafka Topic에 이벤트가 발행되고 알림 서비스가 구독해 로그 출력되는 과정 시연

5. **Q&A (2분)**
   다른 팀 질문에 답변, 구현 중 겪은 이슈 공유

Sprint Review & Retrospective

Sprint Review & Retro · 1/4

- 팀별 구현 결과 데모 (Sprint Review) - 동작하는 기능을 직접 시연
- 계획 대비 실행 회고 (Retrospective) - Day1 계획과 Day2 실제 구현의 차이 회고
- "계획 대비 실행" 갭 분석 및 Q&A
- 다음 슬라이드에서 Review 진행 가이드, 갭 분석 프레임워크, Retro 시나리오를 다룹니다.

Sprint Review 실전 진행 가이드

Sprint Review & Retro · 2/4

단계	소요시간	진행 내용
데모 시연	10분	수강신청 시스템의 전체 흐름(로그인→신청→이벤트)을 직접 실행
산출물 리뷰	5분	인증서버/Eureka/Kafka 각 컴포넌트가 계획대로 동작하는지 확인
이해관계자 피드백	5분	강사 및 타 팀이 개선점·질문을 제시

Review는 "무엇을 완성했는가"를 보여주는 자리이며, 미완성 Task가 있다면 솔직히 공유하고, 다음 계획에 반영합니다.

Day1 계획 vs Day2 실행 갭 분석 프레임워크

Sprint Review & Retro · 3/4

Day1 계획 항목	Day2 실행 결과	차이 원인 / 코멘트
인증/로그인 API 구현	**완료**	예상보다 SOLID 리팩토링에 시간 소요
Eureka 연동	**완료**	계획대로 진행
Kafka 이벤트 발행/구독	**부분 완료**	Consumer 장애 대응 로직은 다음 Sprint로 이월

갭 분석 질문 (Retrospective 전 팀 내 논의)

- 계획보다 오래 걸린 Task는 무엇이고, 원인은 추정 오류였는가 기술적 난이도였는가?
- 다음 Sprint(또는 실제 프로젝트)에서는 어떤 추정 기준을 보완할 것인가?

실행 시나리오 - 전체 Retrospective 진행 대본

Sprint Review & Retro · 4/4

Keep (유지할 것)

- Sprint 단위로 나누니 Kafka 같은 낯선 기술도 부담 없이 접근했다
- Daily Scrum으로 막힌 부분을 빨리 공유할 수 있었다

Problem (문제점)

- Story Point 추정이 실제 구현 난이도보다 낮게 잡혔다
- OAuth 설정 문서를 찾는 데 예상보다 시간이 걸렸다

Try (다음에 시도할 것)

- 다음 프로젝트에서는 Spike (사전 조사) Task를 별도로 배정
- 기술 난이도가 높은 Task는 Point를 보수적으로 추정

Day 2 마무리 · 전체 과정 총정리



백로그 도출부터 Sprint 1~3 실행, Review·Retrospective까지 전체 Agile 사이클을 직접 경험했습니다.

백로그 도출 (Day1) > Sprint1 인증 OAuth > Sprint2 Eureka > Sprint3 Kafka+데모

3. MSA를 이용한 웹서비스 구축

- 요구사항 분석
- 도메인 분리
- 인프라 구성 설계
- 서비스간 통신 설계
- DB 설계 / ERD
- API 명세 설계
- 컨테이너 구성 및 배포

SK 주식회사 AX

MSA 서비스 프로젝트 개발 실습 (온라인 교육 플랫폼 구축)

Step 1. 요구사항 분석

목적 : 전체 기능을 파악하고 서비스 경계 도출의 기반 마련

주체	기능
수강생	회원가입, 로그인, 강의 검색, 수강신청, 결제, 추천 강의 조회
강사	회원가입, 로그인, 강의 등록
시스템	결제 완료 후 수강 활성화, 수강 완료 후 추천 갱신

Step 2. 도메인 분리

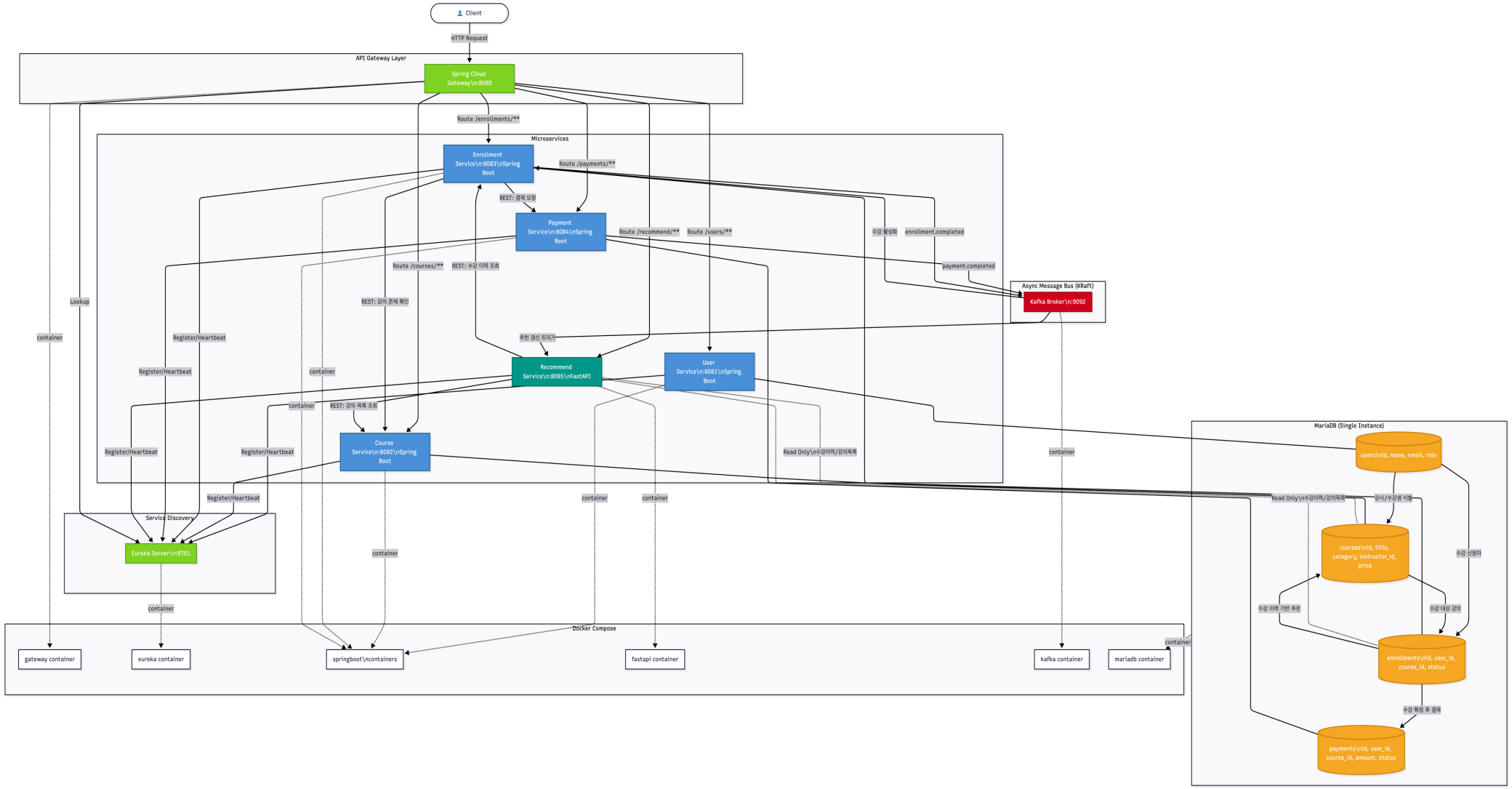
목적 : 기능을 묶어 서비스 경계 직접 도출

서비스	스택	포트	묶인 기능	핵심 데이터
User Service	Spring Boot	8081	회원가입, 로그인, JWT 인증	users
Course Service	Spring Boot	8082	강의 등록, 목록, 검색, 카테고리 관리	courses
Enrollment Service	Spring Boot	8083	수강신청, 수강 상태 관리	enrollments
Payment Service	Spring Boot	8084	결제 처리, 결제 내역	payments
Recommend Service	FastAPI	8085	수강 이력 기반 규칙 추천	enrollments + courses 조회

MSA 서비스 프로젝트 개발 실습 (온라인 교육 플랫폼 구축)

Step 3. 인프라 구성 설계

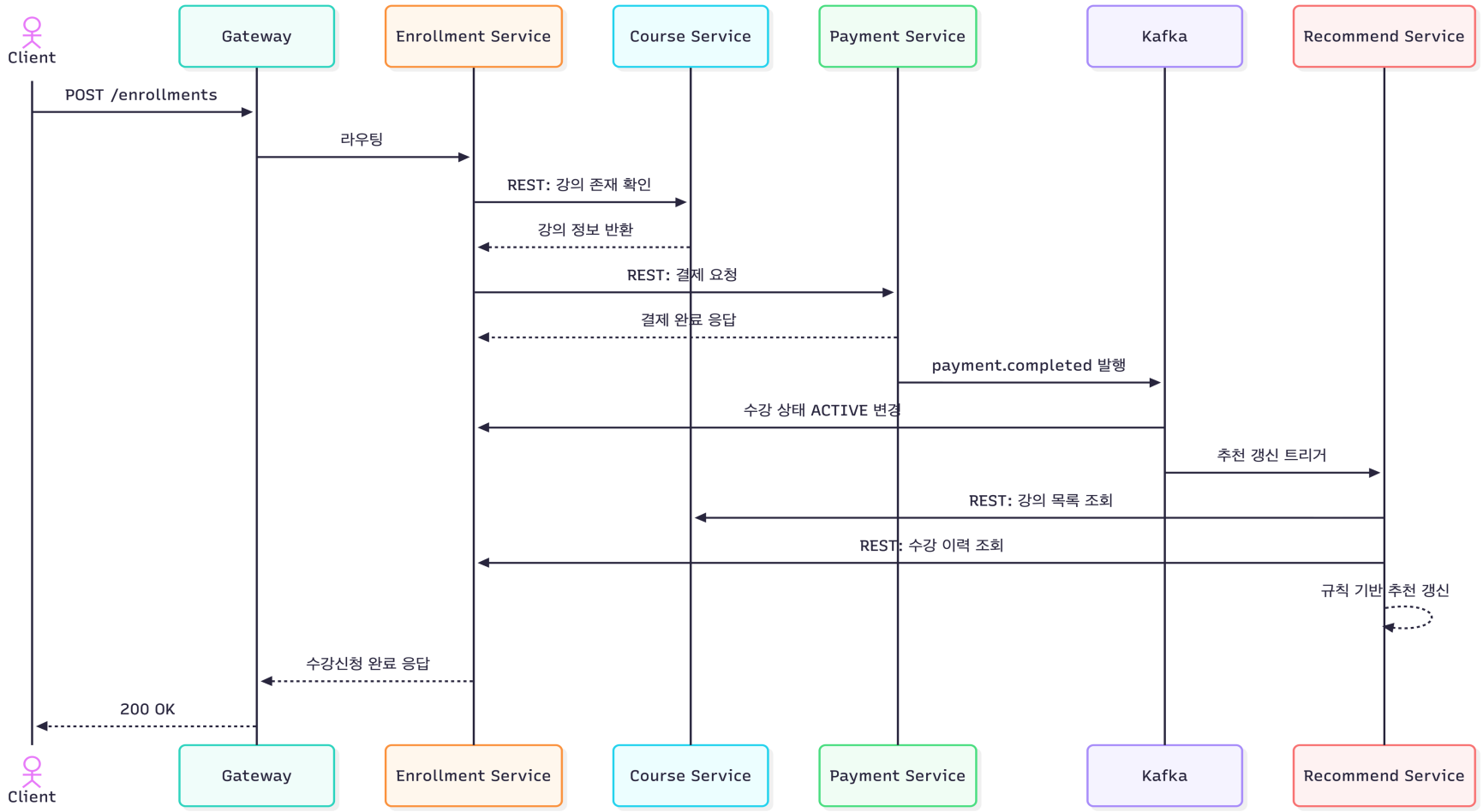
목적 : 서비스를 둘러싼 인프라 구성 요소와 역할 정의



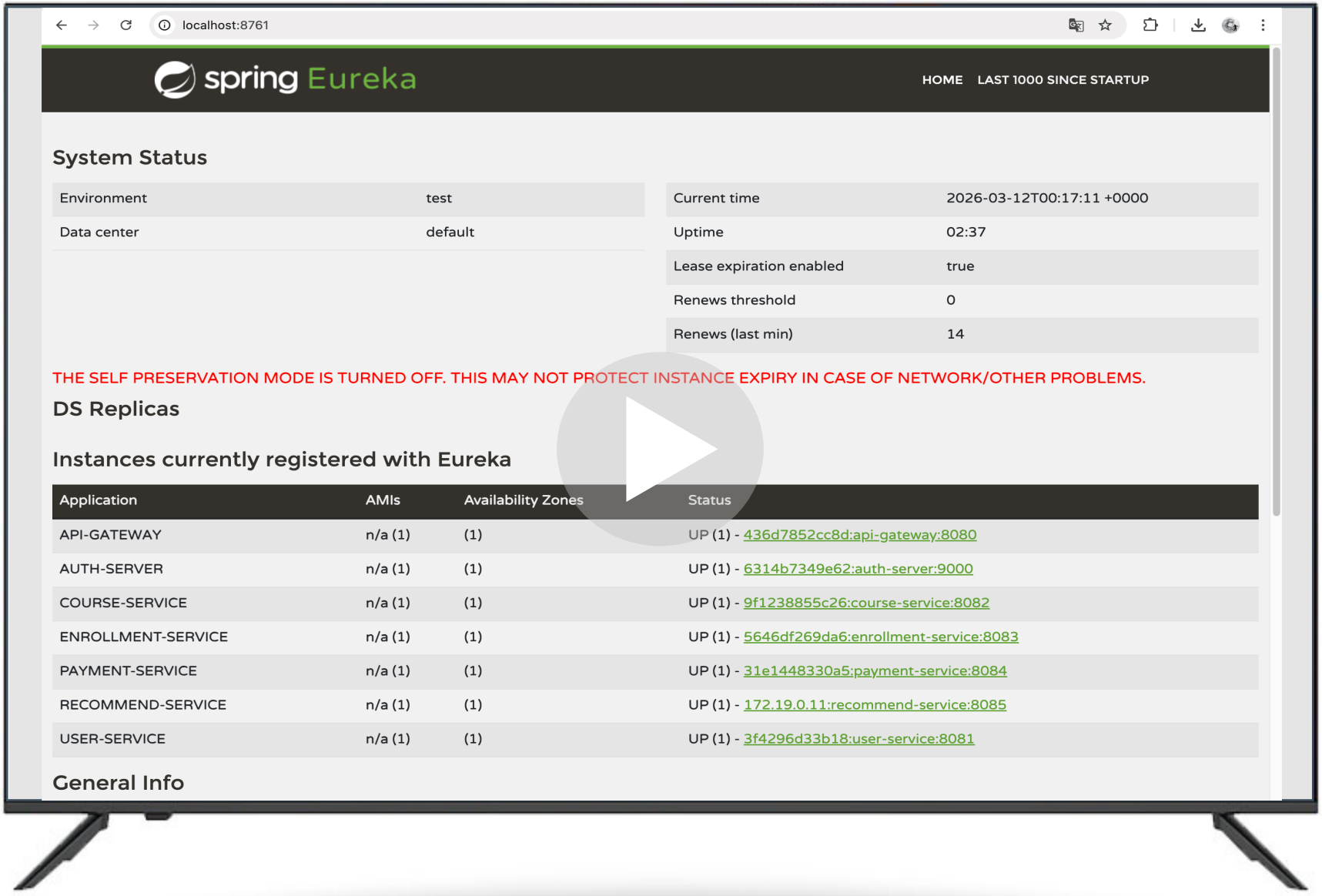
MSA 서비스 프로젝트 개발 실습 (온라인 교육 플랫폼 구축)

Step 4. 서비스간 통신 설계

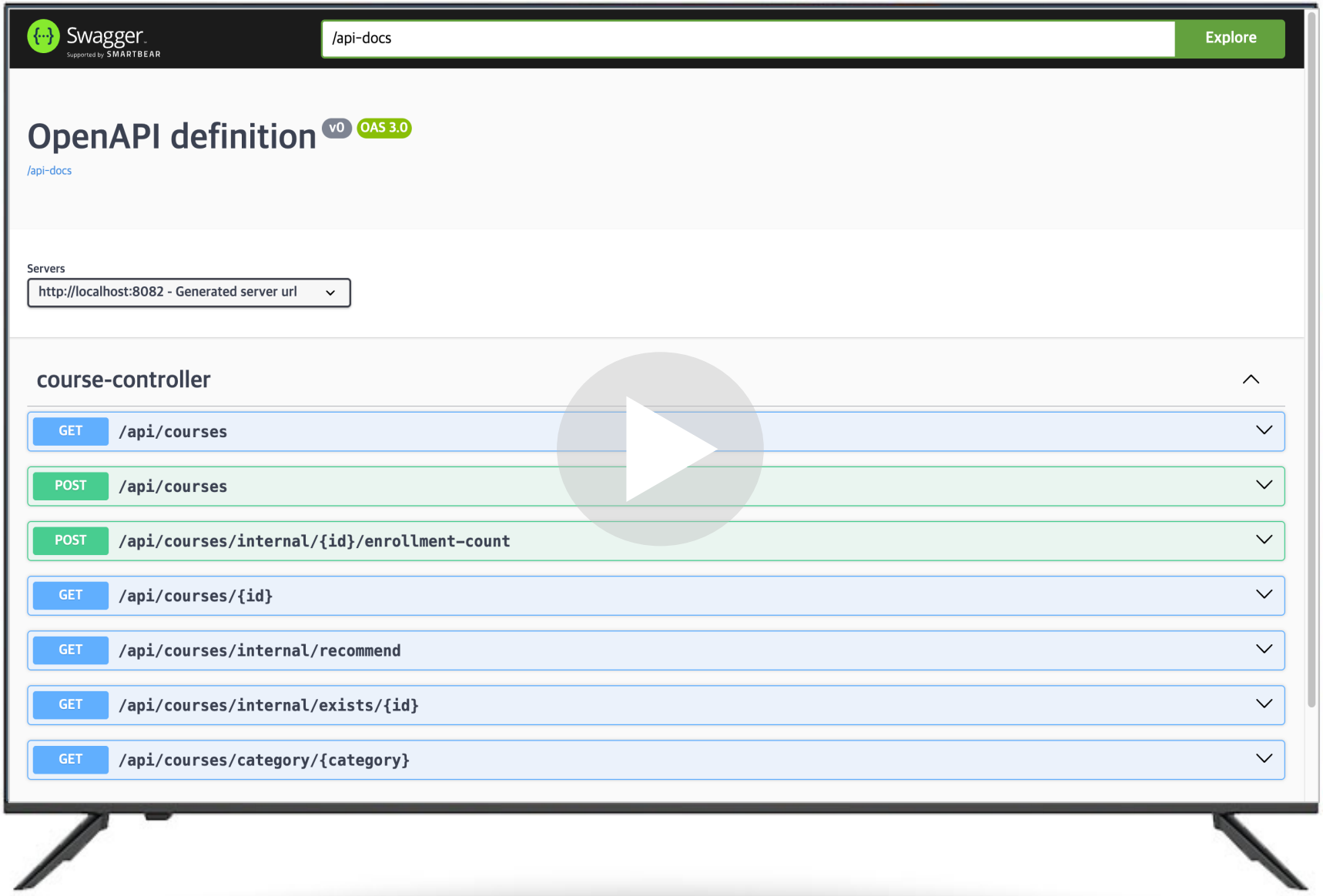
목적 : 동기/비동기 통신 구간 판단



MSA 서비스 프로젝트 개발 실습 (온라인 교육 플랫폼 구축)



MSA 서비스 프로젝트 개발 실습 (온라인 교육 플랫폼 구축)



MSA 서비스 프로젝트 개발 실습 (온라인 교육 플랫폼 구축)

